pySLAM: An Open-Source, Modular, and Extensible Framework for SLAM

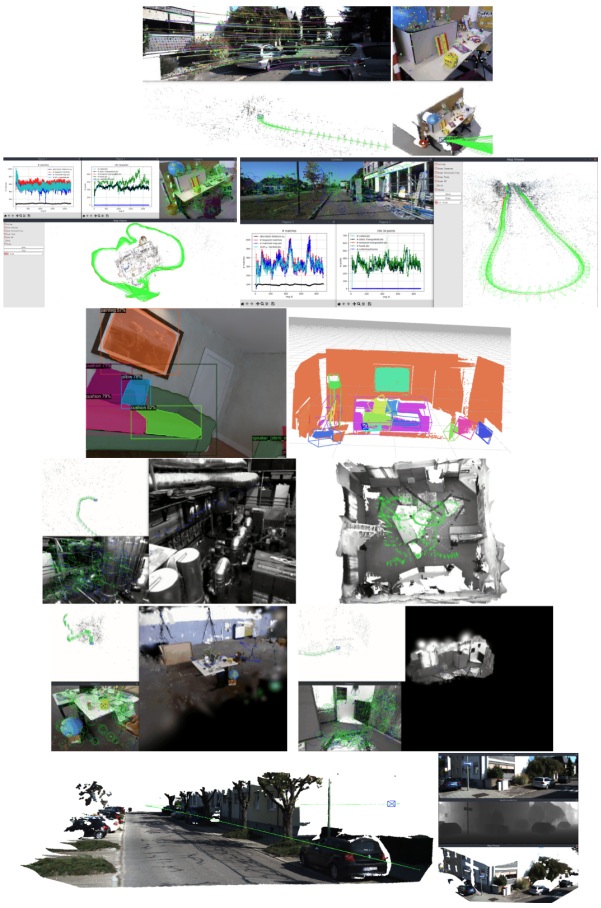
Luigi Freda

October 25, 2026

github.com/luigifreda/pyslam

Abstract

pySLAM is an open-source framework for Visual SLAM that supports monocular, stereo, and RGB-D cameras. The framework integrates a broad range of classical and learning-based local features, multiple loop closure strategies, depth prediction models, semantic segmentation and mapping, a volumetric reconstruction pipeline, and a unified interface to feed-forward multi-view 3D reconstruction models. Multiple camera models are supported. Its sparse SLAM core is implemented in both Python and C++, which lets users trade flexibility for speed without changing the rest of the pipeline. A unified dataset interface, automated evaluation scripts, and a regression test suite support reproducible experiments across more than fifteen dataset types. Designed for both beginners and experienced researchers, pySLAM emphasizes rapid prototyping, extensibility, and reproducibility, and encourages research that bridges traditional and deep learning-based approaches. This document[1] presents the architecture of the framework, its main components, and the tools it provides for evaluation and testing.



[1] You may find an updated version of this document at: github.com/luigifreda/pyslam/blob/master/docs/tex/document.pdf

Contents

1 Introduction **4**

2 Main Features **5**

3 Related Work **5**

4 System Architecture **6**
4.1 Main Scripts . . . 6
4.2 System overview . . . 7
SLAM Workflow and Components . . . 7
4.3 Main SLAM System Components . . . 9
Feature Tracker . . . 9
Feature Matcher . . . 9
Loop Detector . . . 10
Depth Estimator . . . 11
Volumetric Integrator . . . 11
Semantic Mapping . . . 12
4.4 Camera Models . . . 12
4.5 C++ Core for Sparse SLAM . . . 13
4.6 3D Scene from Views . . . 14

5 Supported components and models **15**
5.1 Supported local features . . . 15
5.2 Supported matchers . . . 17
5.3 Supported global descriptors and local descriptor aggregation methods . . . 17
5.4 Supported depth prediction models . . . 18
5.5 Supported volumetric mapping methods . . . 18
5.6 Supported semantic segmentation methods . . . 18
Supported models for feed-forward inference of 3D scenes from multiple images . . . 18

6 Experimental Evaluation **18**
6.1 Run a SLAM evaluation . . . 18
6.2 TUM Evaluation . . . 19
6.3 EUROC Evaluation . . . 20
6.4 KITTI Evaluation . . . 21
6.5 pySLAM performances and comparative evaluations . . . 22
6.6 Automated Testing . . . 22
6.7 Camera Models on TUM-VI . . . 22

7 Conclusions and Future Work **22**

Acknowledgments **23**

A Usage **29**
A.1 Visual odometry . . . 29
A.2 Full SLAM . . . 29
A.3 Selecting a dataset and different configuration parameters . . . 29
A.4 Feature tracking . . . 29
A.5 Loop closing . . . 30
A.6 Volumetric reconstruction . . . 31
Depth prediction . . . 32
Semantic mapping and Image Segmentation . . . 32
A.7 Saving and reloading . . . 34
Save a map . . . 34
Reload a saved map and relocalize in it . . . 34
Trajectory saving . . . 34
Optimization engines . . . 34
A.8 SLAM GUI . . . 34
A.9 Monitor the logs for tracking, local mapping, and loop closing simultaneously . . . 35
A.10 Scene from Views . . . 35

B Configuration **35**
B.1 Main configuration file . . . 35

B.2 Datasets . . . 36
KITTI Datasets . . . 36
TUM Datasets . . . 36
TUM-VI Datasets . . . 37
ICL-NUIM Datasets . . . 37
EuRoC Datasets . . . 37
Replica Datasets . . . 37
Tartanair Datasets . . . 37
7 Scenes dataset . . . 37
Neural RGBD dataset . . . 37
Rover dataset . . . 37
ScanNet Datasets . . . 38
CLIO Datasets . . . 38
ROS1 bags . . . 38
ROS2 bags . . . 38
MCAP file . . . 38
Video and Folder Datasets . . . 38
B.3 Camera Settings . . . 39

1 Introduction

pySLAM is an open-source framework designed to accelerate research and development in Visual SLAM (Simultaneous Localization and Mapping)[20] and Spatial AI[26]. It originated from a practical need identified during the development of a SLAM/Spatial AI course: The absence of a unified, accessible, and extensible SLAM ecosystem in C++ and Python. The project was driven by several foundational motivations, including the desire to rapidly prototype and evaluate both *classical* and *learning-based* features within complete VO/SLAM pipelines, ensure reproducibility across a wide range of datasets, and significantly lower the entry barrier for newcomers to the field.

A key decision behind pySLAM development was the choice of *Python* as its primary language. Compared to C++, Python enables faster experimentation and benefits from a mature and expanding ecosystem of deep learning tools and research-oriented APIs. This choice aligns with current trends in computer vision and robotics research, where rapid prototyping and flexible experimentation are essential.

However, designing a modern SLAM framework in Python presents several challenges. The SLAM landscape is highly fragmented – across feature types, datasets, programming languages (Python vs. C++), and development environments (virtual environments, Docker, versioning issues). Deep learning integration further complicates matters due to backend fragmentation (e.g., PyTorch vs. TensorFlow). These inconsistencies often lead to reproducibility bottlenecks and high barriers to experimental research, where setting up a functional baseline can become more laborious than the research itself.

To address these issues, pySLAM is built on several key design principles. *Modularity* is central: components such as local/global features[106, 44, 62], loop closure mechanisms[91], semantic mapping[36, 22, 88], and volumetric integration[29, 32] can be easily swapped or extended. This architecture supports the use of interchangeable baselines, enabling researchers to plug-and-play with different configurations depending on their goals. *Extensibility* is facilitated through a base-class architecture that allows new components to be integrated with minimal friction, making pySLAM a flexible platform for benchmarking and algorithm development.

In terms of usability, pySLAM emphasizes simplicity in the build and deployment process. It supports a unified Python environment compatible with *conda*, *pyenv*, and *pixi*, and offers a one-command installation process that covers setup and dataset acquisition. Cross-platform compatibility (Linux, macOS, Windows with WSL2) and full source accessibility further enhance its utility for both academic and industrial use.

Another cornerstone of the framework is standardized dataset handling. By providing a unified interface to multiple datasets, pySLAM encourages *reproducibility* and *reusability*. It supports consistent data loading, map state saving/loading, and automatic generation of evaluation metrics – including final and online trajectory estimation – thus enabling robust comparison between algorithms.

Functionally, pySLAM supports the full visual SLAM pipeline with monocular, stereo, and RGB-D configurations. It accommodates both traditional local features and deep-learning-based methods, and includes loop closure detection integrated with optimization backends like g2o[50] and GTSAM[27]. The framework also supports 3D reconstruction via different voxel grid models, TSDF[29] and emerging techniques like Gaussian Splatting[9], and integrates deep learning models for depth prediction and semantic scene understanding.

Beyond SLAM, pySLAM also provides a unified *ScenefromViews* architecture for modern feed-forward inference of 3D scenes from multiple views. The module supports both pairwise models (DUSt3R[112], MASt3R[51]) that process image pairs and perform global alignment, as well as multi-view models (Depth Anything V3[56], MV-DUSt3R[99], VGGT[111], robust VGGT[38], and Fast3R[116]) that process multiple views simultaneously in a single forward pass. This modular design enables flexible integration of diverse 3D reconstruction techniques while maintaining a consistent API across different model architectures.

Together, these capabilities position pySLAM as a powerful research toolkit – aiming to be a "Swiss army knife" for SLAM development, evaluation, and education. By streamlining reproducibility, supporting modular experimentation, and lowering the technical threshold for entry, pySLAM addresses some of the most persistent challenges in contemporary SLAM research.

The remainder of this document is organized as follows. Section 2 summarizes the main features of the framework, and Section 3 relates pySLAM to prior SLAM systems and to the methods it integrates. Section 4 describes the system architecture: the processing modules, their main components, the camera models, the C++ core, and the Scene from Views module. Section 5 lists the supported components and models. Section 6 presents the evaluation tools, the results obtained on TUM, EuRoC, and KITTI, the automated test suite, and an initial validation of the camera models on TUM-VI. Section 7 concludes and outlines future work. Appendix A describes how to run the main tools, and Appendix B how to configure datasets and cameras.

2 Main Features

pySLAM provides a python implementation of a *Visual SLAM* pipeline that supports **monocular**, **stereo** and **RGBD** cameras. It offers the following **features** in a single python environment:

- A wide range of classical and modern **local features** with a convenient interface for their integration.
- Various loop closing methods, including **descriptor aggregators** such as visual Bag of Words (BoW[33], iBow[34]), Vector of Locally Aggregated Descriptors (VLAD[41]), and modern **global descriptors** (image-wise descriptors).
- A **volumetric reconstruction pipeline** that processes available depth and color images with volumetric integration and provides an output dense reconstruction. This can use **TSDF**[29] with voxel hashing or incremental **Gaussian Splatting**[66, 46].
- Integration of **depth prediction models** within the SLAM pipeline. These include DepthPro[17], DepthAnythingV2[100], RAFT-Stereo[101], CREStereo[55], MASt3R[52], MVDUSt3R[98], etc.
- A suite of segmentation models for **semantic understanding** of the scene, such as DeepLabv3[23], Segformer[114], CLIP[57], EOV-Seg[75], DETIC[117], ODISE[115], RFDETR[83], and YOLO[79, 80, 81, 18, 43, 110, 42, 86].
- A **Scene from Views** module providing a unified interface for feed-forward inference of 3D scenes from multiple views, supporting models such as DUSt3R[112], MASt3R[51], Depth Anything V3[56], MV-DUSt3R[99], VGGT[111], and Fast3R[116].
- A modular sparse SLAM **Core**, implemented both in Python and C++ (with custom pybind11 bindings), allowing users to switch between high-performance/speed and high-flexibility modes.
- Several **camera models** in the sparse pipeline: pinhole, Kannala–Brandt[45], rational distortion, and double sphere[108], with native and rectified fisheye stereo on TUM-VI[90].
- Additional tools for VO (Visual Odometry) and SLAM, with built-in support for both **g2o**[50] and **GTSAM**[27], along with custom Python bindings for features not available in the original libraries
- Built-in support for more than **15 dataset types**. The list is currently growing.
- Tools for **reproducible evaluation**: automated multi-dataset evaluation, a regression test suite, and an end-to-end replay harness with performance and accuracy gates.

pySLAM serves as a flexible baseline framework to experiment with VO/SLAM techniques, *local features*, *descriptor aggregators*, *global descriptors*, *volumetric integration*, *depth prediction*, *semantic mapping*, *scene from views*, and *C++ core*. It allows to explore, prototype and develop VO/SLAM pipelines. pySLAM is a research framework and a work in progress. It is not optimized for real-time performance.

3 Related Work

Feature-based visual SLAM. PTAM [48] introduced the separation of camera tracking and keyframe-based mapping into parallel threads, with bundle adjustment running in the background. ORB-SLAM2 [72] built on this design and added loop closing, relocalization, and support for monocular, stereo, and RGB-D cameras, using ORB features [85] and a bag of binary words [33] for place recognition. ORB-SLAM3 [21] extended the system to visual-inertial and multi-map SLAM and abstracted the camera model, so that pinhole and fisheye cameras share the same pipeline. pySLAM adopts the same tracking, local mapping, and loop closing decomposition (Sect. 4), but implements it as a modular framework in which each component, from the local features to the optimization backend, can be replaced through a factory interface. A broader overview of the SLAM problem and its open challenges is given in [20].

Place recognition and loop closing. Loop detection has moved from aggregations of local descriptors, such as bags of words [33] and VLAD [41], to global image descriptors learned end to end, such as NetVLAD [5], CosPlace [15], EigenPlaces [16], and MegaLoc [14]. Schubert et al. [91] provide a tutorial on visual place recognition. pySLAM exposes both families behind the same loop detector interface, so they can be compared within a complete SLAM pipeline rather than in isolation.

Dense, semantic, and learned reconstruction. Dense reconstruction from RGB-D data has been surveyed in [118]. pySLAM integrates TSDF fusion with voxel hashing [29], incremental Gaussian Splatting [66, 46], and semantic volumetric mapping, following earlier work on combining sparse SLAM with dense and semantic mapping [32, 70]. More recently, feed-forward models such as DUSt3R [112], MASt3R [51], and VGGT [111] infer

camera poses and 3D structure directly from a set of images. pySLAM offers them through a single *Scene from Views* interface (Sect. 4.6).

Camera models. Wide-angle and fisheye lenses are commonly described by the Kannala–Brandt model [45] or by the double sphere model [108], and the TUM-VI benchmark [90] provides fisheye stereo sequences with motion-capture ground truth. Sect. 4.4 describes how these models are integrated into the sparse pipeline of pySLAM.

4 System Architecture

4.1 Main Scripts

A convenient entry point is the following collection of **main scripts**:

- `main_vo.py` combines the simplest VO ingredients without performing any image point triangulation or windowed bundle adjustment. At each step k, `main_vo.py` estimates the current camera pose C_k with respect to the previous one C_{k-1}. The inter-frame pose estimation returns $[R_{k-1,k}, t_{k-1,k}]$ with $\|t_{k-1,k}\| = 1$. With this very basic approach, you need to use a ground truth in order to recover a correct inter-frame scale s and estimate a valid trajectory by composing $C_k = C_{k-1}[R_{k-1,k}, st_{k-1,k}]$. This script is a first start to understand the basics of inter-frame feature tracking and camera pose estimation.
- `main_slam.py` adds feature tracking along multiple frames, point triangulation, keyframe management, bundle adjustment, loop closing, dense mapping and depth inference in order to estimate the camera trajectory and build both a sparse and dense map. It's a full SLAM pipeline and includes all the basic and advanced blocks which are necessary to develop a real visual SLAM pipeline.
- `main_feature_matching.py` shows how to use the basic feature tracker capabilities (*feature detector* + *feature descriptor* + *feature matcher*) and allows to test the different available local features.
- `main_depth_prediction.py` shows how to use the available depth inference models to get depth estimations from input color images.
- `main_semantic_image_segmentation.py` runs the configured semantic segmentation model on each frame of the selected dataset and visualizes the extracted semantic information.
- `main_map_viewer.py` reloads a saved map and visualizes it. Further details on how to save a map here.
- `main_map_dense_reconstruction.py` reloads a saved map and uses a configured volumetric integrator to obtain a dense reconstruction (see here).
- `main_slam_evaluation.py` enables automated SLAM evaluation by executing `main_slam.py` across a collection of datasets and configuration presets (see here).
- `main_scene_from_views.py` infers 3D scenes from multiple images using models like Mast3r, VGGT, DepthFromAnythingV3, and Fast3R (see here).

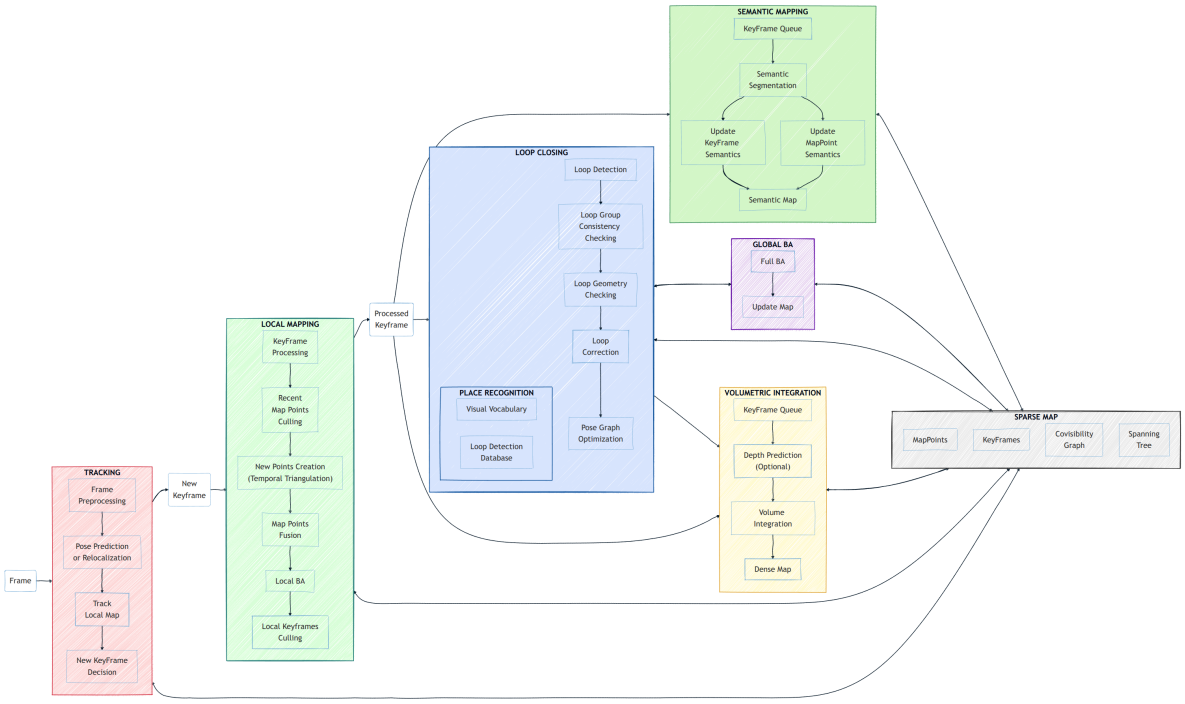


Figure 1: SLAM workflow.

4.2 System overview

This section presents some diagram sketches that provide an overview of the main workflow, system components, and class relationships/dependencies. To make the diagrams more readable, some minor components and arrows have been omitted.

SLAM Workflow and Components

Fig. 1 illustrates the SLAM workflow, which is composed of six main parallel processing modules:

- *Tracking*: estimates the camera pose for each incoming frame by extracting and matching local features to the local map, followed by minimizing the reprojection error through motion-only Bundle Adjustment (BA). It includes components such as pose prediction (or relocalization), feature tracking, local map tracking, and keyframe decision-making.
- *Local Mapping*: updates and refines the local map by processing new keyframes. This involves culling redundant map points, creating new points via temporal triangulation, fusing nearby map points, performing Local BA, and pruning redundant local keyframes.
- *Loop Closing*: detects and validates loop closures to correct drift accumulated over time. Upon loop detection, it performs loop group consistency checks and geometric verification, applies corrections, and then launches Pose Graph Optimization (PGO) followed by a full Global Bundle Adjustment (GBA). Loop detection itself is delegated to a parallel process, the *Loop Detector*, which operates independently for better responsiveness and concurrency.
- *Global Bundle Adjustment*: triggered by the Loop Closing module after PGO, this step globally optimizes the trajectory and the sparse structure of the map to ensure consistency across the entire sequence.
- *Volumetric Integration*: uses the keyframes, with their estimated poses and back-projected point clouds, to reconstruct a dense 3D map of the environment. This module optionally integrates predicted depth maps and maintains a volumetric representation such as a TSDF [29] or incremental Gaussian Splatting-based volume [66, 46].
- *Semantic Mapping*: enriches the SLAM map with dense semantic information by applying pixel-wise segmentation to selected keyframes. Semantic predictions are fused across views to assign semantic labels or descriptors to keyframes and map points. The module operates in parallel, consuming keyframes and associated image data from a queue, applying a configured semantic segmentation model, and updating the map with fused semantic features. This enables advanced downstream tasks such as semantic navigation, scene understanding, and category-level mapping.

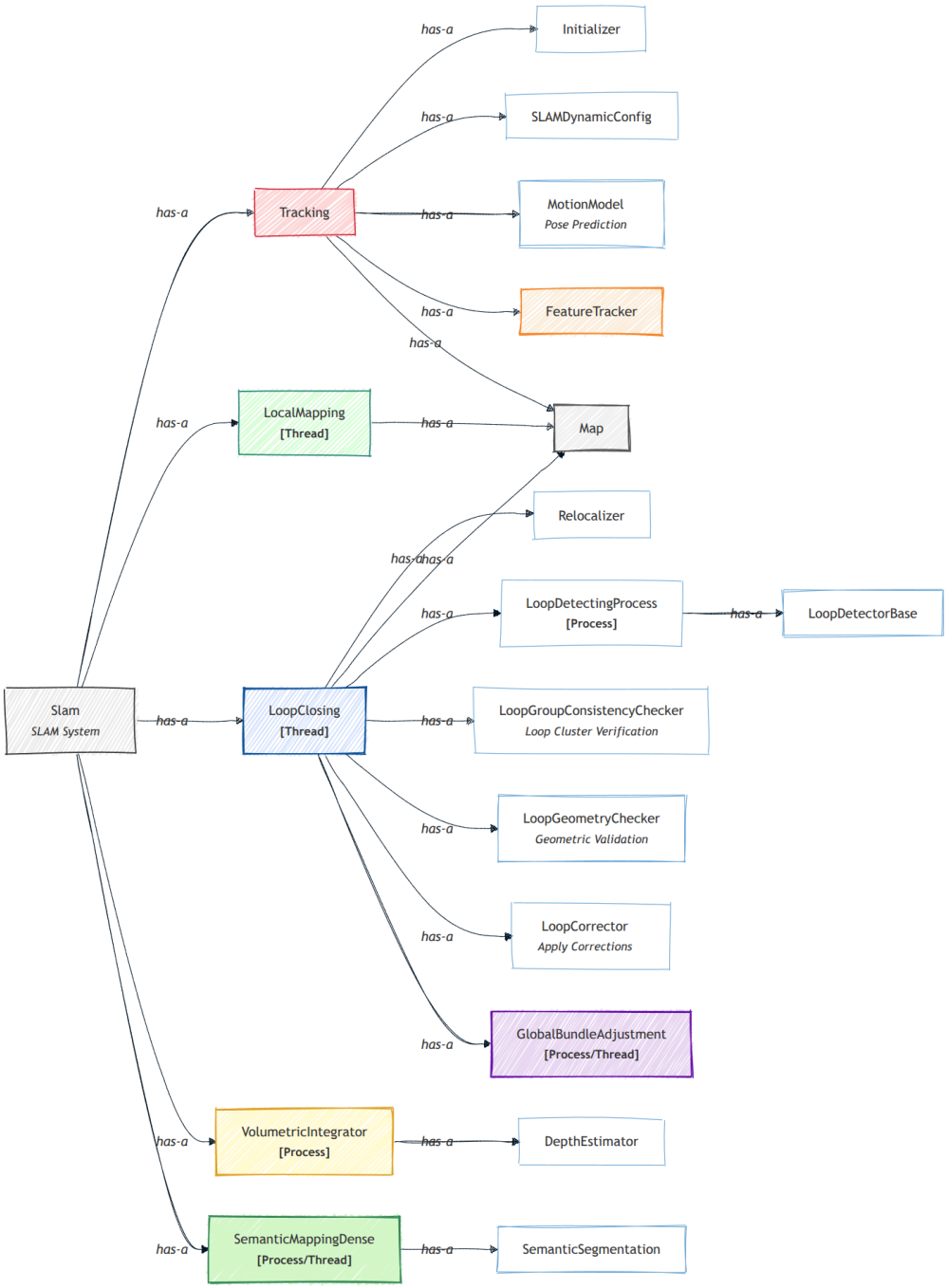


Figure 2: SLAM components.

The first four modules follow the established PTAM [48] and ORB-SLAM [72] paradigm. Here, the *Tracking* module serves as the front-end, while the remaining modules operate as part of the back-end.

In parallel, the system constructs two types of maps:

- a *sparse map* $\mathcal{M}_s = (\mathcal{K}, \mathcal{P})$, composed of a set of keyframes $\mathcal{K}$ and 3D points $\mathcal{P}$ derived from matched features;
- a *volumetric map* (or dense map) $\mathcal{M}_v$, constructed by the Volumetric Integration module, which fuses back-projected point clouds from the keyframes $\mathcal{K}$ into a dense 3D model.

To ensure consistency between the sparse and volumetric representations, the volumetric map is updated or re-integrated whenever global pose adjustments occur (e.g., after loop closures).

Fig. 2 details the internal components and interactions of the above modules. In certain cases, **processes** are

employed instead of **threads**. This is due to Python's Global Interpreter Lock (GIL), which prevents concurrent execution of multiple threads in a single process. The use of multiprocessing circumvents this limitation, enabling true parallelism at the cost of some inter-process communication overhead (e.g., via pickling). For an insightful discussion, see this related post.

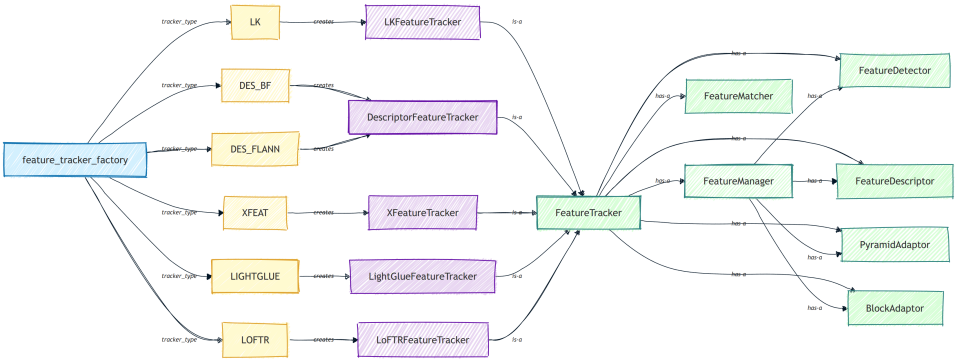


Figure 3: Feature tracker.

4.3 Main SLAM System Components

Feature Tracker

The *Feature Tracker* consists of the following key sub-components:

- *Feature Manager*: Manages local feature operations. It includes the `FeatureDetector`, `FeatureDescriptor`, and adaptors for pyramid management and image tiling.
  - *Feature Detector*: Identifies salient and repeatable keypoints in the image, such as corners or blobs, which are likely to be robust under viewpoint and illumination changes.
  - *Feature Descriptor*: Computes a distinctive descriptor for each detected keypoint, encoding its local appearance to enable robust matching across frames. Examples include ORB [85], SIFT [59], or SuperPoint [28] descriptors.
- *Feature Matcher*: Establishes correspondences between features in successive frames (or stereo pairs) by comparing their descriptors or directly inferring matches from image content. Matching can be performed using brute-force, k-NN with ratio test, or learned matching strategies. Refer to Sect. 4.3 for further details.

Sect. 5.1 lists the supported local feature extractors and detectors.

The diagram in Fig. 3 presents the architecture of the *Feature Tracker* system. It is structured around a `feature_tracker_factory`, which instantiates specific tracker types such as `LK`, `DES_BF`, `DES_FLANN`, `XFEAT`[12], `LIGHTGLUE`[58], and `LOFTR`[97]. Each tracker type creates a corresponding implementation (e.g., `LKFeatureTracker`, `DescriptorFeatureTracker`, etc.), all of which inherit from a common `FeatureTracker` interface.

The `FeatureTracker` class is composed of several key sub-components, including a `FeatureManager`, `FeatureDetector`, `FeatureDescriptor`, `PyramidAdaptor`, `BlockAdaptor`, and `FeatureMatcher`. The `FeatureManager` also encapsulates instances of the detector, descriptor, and adaptors, highlighting the modular and reusable design of the tracking pipeline.

Feature Matcher

The diagram in Fig. 4 illustrates the architecture of the *Feature Matcher* module. At its core is the `feature_matcher_factory`, which instantiates matchers based on a specified `matcher_type`, such as `BF`, `FLANN`, `XFEAT`[12], `LIGHTGLUE`[58], and `LOFTR`[97]. Each of these creates a corresponding matcher implementation (e.g., `BfFeatureMatcher`, `FlannFeatureMatcher`, etc.), all inheriting from a common `FeatureMatcher` interface.

The `FeatureMatcher` class encapsulates several configuration parameters and components, including the matcher engine (`cv2.BFMatcher`, `FlannBasedMatcher`, `xfeat.XFeat`, etc.), as well as the `matcher_type`, `detector_type`, `descriptor_type`, `norm_type`, and `ratio_test` fields. This modular structure supports extensibility and facilitates switching between traditional and learning-based feature matching backends.

The Section 5.2 reports a list of supported feature matchers.

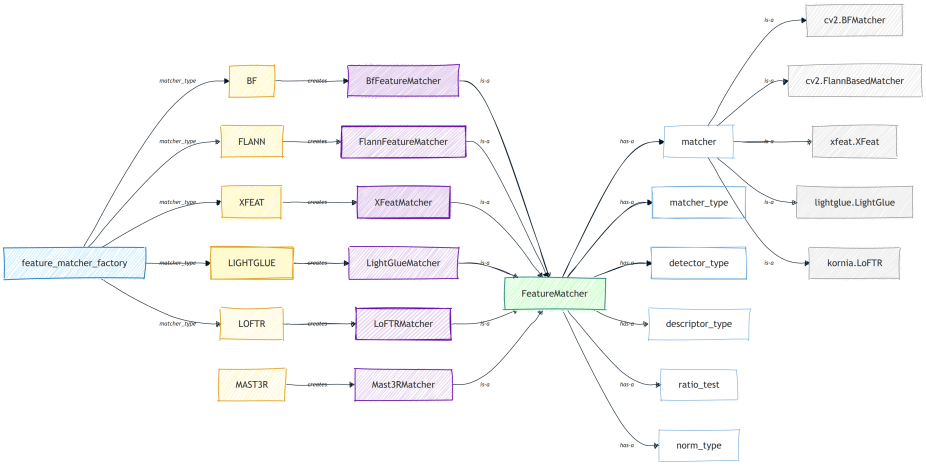


Figure 4: Feature matcher.

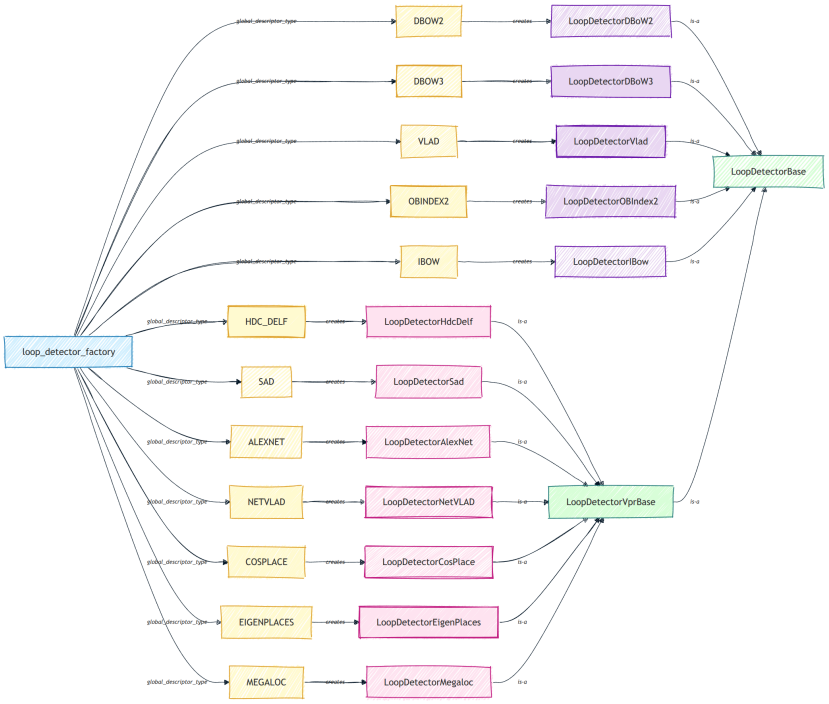


Figure 5: Loop detector.

Loop Detector

The diagram in Fig. 5 shows the architecture of the *Loop Detector* component. A central `loop_detector_factory` instantiates loop detectors based on the selected `global_descriptor_type`, which may include traditional descriptors (e.g., `DBOW2`[33], `VLAD`[41], `IBOW`[34]) or deep learning-based embeddings (e.g., `NetVLAD`[5], `CosPlace`[15], `EigenPlaces`[16]).

Each descriptor type creates a corresponding loop detector implementation (e.g., `LoopDetectorDBoW2`, `LoopDetectorNetVLAD`), all of which inherit from a base class hierarchy. Traditional methods inherit directly from `LoopDetectorBase`, while deep learning-based approaches inherit from `LoopDetectorVprBase`, which itself extends `LoopDetectorBase`. This design supports modular integration of diverse place recognition techniques within a unified loop closure framework.

The Section 5.3 reports a list of supported loop closure methods with the adopted global descriptors and local descriptor aggregation methods.

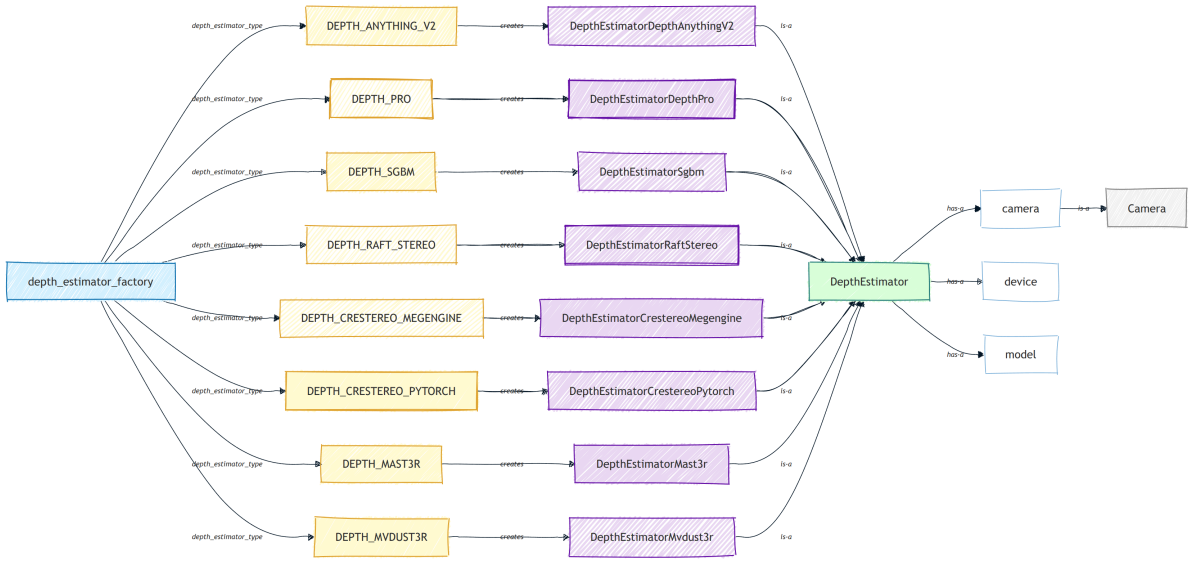


Figure 6: Depth estimator.

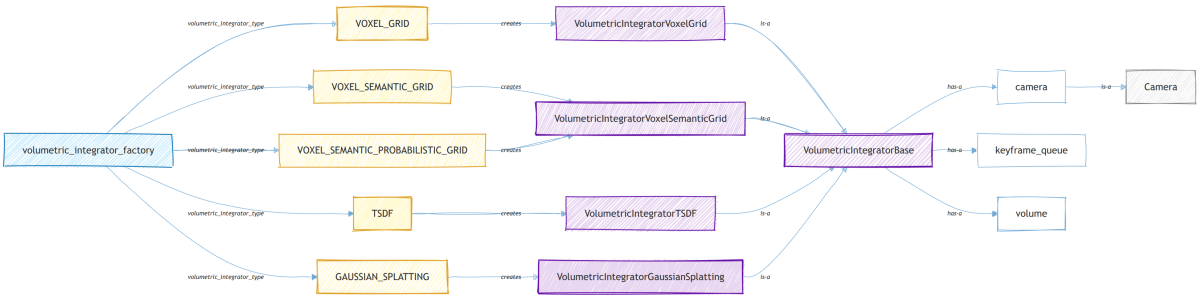


Figure 7: Volumetric integrator.

Depth Estimator

The diagram in Fig. 6 illustrates the architecture of the *Depth Estimator* module. A central `depth_estimator_factory` creates instances of various depth estimation backends based on the selected `depth_estimator_type`, including both traditional and learning-based methods such as `DEPTH_SGBM`[40], `DEPTH_RAFT_STEREO`[101], `DEPTH_ANYTHING_V2`[100], `DEPTH_MAST3R`[52], and `DEPTH_MVDUST3R`[98].

Each estimator type instantiates a corresponding implementation (e.g., `DepthEstimatorSgbm`, `DepthEstimator-CrestereoMegengine`, etc.), all inheriting from a common `DepthEstimator` interface. This base class encapsulates shared dependencies such as the `camera`, `device`, and `model` components, allowing for modular integration of heterogeneous depth estimation techniques across stereo, monocular, and multi-view pipelines.

The Section 5.4 reports a list of supported depth estimation/prediction models.

Volumetric Integrator

The diagram in Fig. 7 illustrates the structure of the *Volumetric Integrator* module. At its core, the `volumetric_integrator_factory` generates specific volumetric integrator instances based on the selected `volumetric_integrator_type`, such as `VOXEL_GRID`, `VOXEL_SEMANTIC_GRID`, `VOXEL_SEMANTIC_PROBABILISTIC_GRID`, `TSDF`[29], and `GAUSSIAN_SPLATTING`[66, 46].

Each type instantiates a dedicated implementation (e.g., `VolumetricIntegratorVoxelGrid`, `VolumetricIntegrator-VoxelSemanticGrid`, `VolumetricIntegratorTSDF`, `VolumetricIntegratorGaussianSplatting`), which inherits from a common `VolumetricIntegratorBase`. This base class encapsulates key components including the `camera`, a `keyframe_queue`, and the `volume`, enabling flexible integration of various 3D reconstruction methods within a unified pipeline.

The Section 5.5 reports a list of supported volume integration methods.

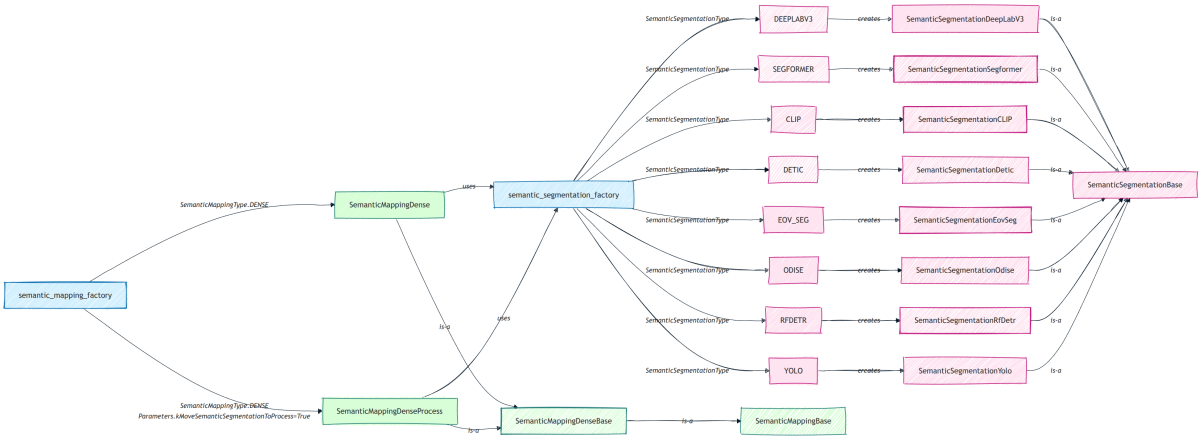


Figure 8: Semantic Mapping.

Semantic Mapping

The diagram in Fig. 8 outlines the architecture of the *Semantic Mapping* module. At its core is the `semantic_mapping_factory`, which creates semantic mapping instances according to the selected `semantic_mapping_type`. Currently, the supported type is `DENSE`, which instantiates either the `SemanticMappingDense` class or the `SemanticMappingDenseProcess` class if

```
Parameters.kSemanticMappingMoveSemanticSegmentationToSeparateProcess=True
```

Both these classes extend `SemanticMappingDenseBase` and runs asynchronously in a dedicated thread to process keyframes as they become available. In particular, the `SemanticMappingDenseProcess` class offloads semantic segmentation inference to a separate process, thereby making better use of CPU core parallelism.

Both `SemanticMappingDense` and `SemanticMappingDenseProcess` integrate semantic information into the SLAM map by leveraging per-frame predictions from a semantic segmentation model. The segmentation model is instantiated via the `semantic_segmentation_factory`, based on the selected `semantic_segmentation_type`. Supported segmentation backends include `DEEPLABV3`[23], `SEGFORMER`[114], `CLIP`[57], `EOV_SEG`[75], `DETIC`[117], `ODISE`[115], `YOLO`[79, 80, 81, 18, 43, 110, 42, 86], and `RFDETR`[83], each of which corresponds to a dedicated class (`SemanticSegmentationDeepLabV3`, `SemanticSegmentationSegformer`, `SemanticSegmentationCLIP`, `SemanticSegmentationEovSeg`, `SemanticSegmentationDetic`, `SemanticSegmentationOdise`), `SemanticSegmentationRfDetr`), `SemanticSegmentationYolo`), all inheriting from the shared `SemanticSegmentationBase`.

The system supports multiple semantic feature representations – such as categorical labels, probability vectors, and high-dimensional feature embeddings – and fuses them into the map using configurable methods like count-based fusion, Bayesian fusion, or feature averaging.

This modular design decouples semantic segmentation from mapping logic, enabling flexible combinations of segmentation models, datasets (e.g., `NYU40`[73], `Cityscapes`[24]), and fusion strategies. It also supports customization via configuration files or programmatic APIs for dataset-specific tuning or deployment.

The Section A.6 provides a list of supported semantic segmentation methods. A more in-depth presentation of the semantic mapping module is provided in [70].

4.4 Camera Models

The sparse pipeline accesses the camera only through its projection $\pi(\mathbf{X}_c)$, its unprojection $\pi^{-1}(\mathbf{u})$, and the analytic Jacobian $\partial\pi/\partial\mathbf{X}_c$ of the projection with respect to a point $\mathbf{X}_c = (x, y, z)^\top$ in the camera frame. Four models are available and are selected by the `Camera.type` field of the settings file. A settings file without that field keeps the linear pinhole model, so existing calibration files are unaffected.

Pinhole. The default model is

$$u = f_x \frac{x}{z} + c_x, \qquad v = f_y \frac{y}{z} + c_y. \tag{1}$$

The distortion coefficients are used only to undistort the detected keypoints, and bundle adjustment compares the linear projection with the undistorted keypoints.

Kannala–Brandt. The fisheye model of [45] with four distortion coefficients (`kannala_brandt4`) maps the angle θ between the ray and the optical axis to a distorted angle θ_d:

$$r = \sqrt{x^2 + y^2}, \quad \theta = \operatorname{atan2}(r, z), \quad \theta_d = \theta \left(1 + k_1\theta^2 + k_2\theta^4 + k_3\theta^6 + k_4\theta^8\right), \tag{2}$$

$$u = f_x\, \theta_d\, \frac{x}{r} + c_x, \qquad v = f_y\, \theta_d\, \frac{y}{r} + c_y. \tag{3}$$

This is the polynomial used by OpenCV's fisheye module. Because the field of view can exceed 180°, a point with $z < 0$ can still have a valid projection.

Rational pinhole. This model (`pinhole_radtan8`) distorts the normalized point $(x/z, y/z)$ with the OpenCV rational model, whose eight coefficients are $k_1, k_2, p_1, p_2, k_3, k_4, k_5, k_6$, and then multiplies it by the intrinsic matrix.

Double sphere. The model of [108] (`double_sphere`) projects a point through two unit spheres whose centers are offset by ξ, followed by a pinhole whose image plane is shifted by α:

$$d_1 = \|\mathbf{X}_c\|, \quad d_2 = \sqrt{x^2 + y^2 + (\xi d_1 + z)^2}, \quad u = f_x \frac{x}{\alpha d_2 + (1-\alpha)(\xi d_1 + z)} + c_x, \tag{4}$$

and analogously for v.

Measurements. For the pinhole model, tracking and optimization use undistorted keypoints, as in ORB-SLAM2 [72]. For the three non-linear models, the projection predicts the detected pixel directly, so projection search and bundle adjustment measure the raw keypoints. For every model the framework also keeps the pinhole image of each keypoint ray, which is used by the fundamental-matrix test and by rectified stereo matching. A stereo observation is the triple (u_l, v_l, u_r). With a physical stereo rig, u_r is predicted by projecting the point through the calibrated right camera and its extrinsics. With an RGB-D sensor it is the virtual coordinate $u_r = f_x x/z + c_x - bf_x/z$, where bf_x is the stereo baseline times the focal length.

Native fisheye stereo. Rectified-row stereo matching assumes a pinhole image. For the non-linear models, stereo correspondences are instead searched on unit bearings: each left bearing defines an epipolar plane in the right camera, and a right keypoint is a candidate when its bearing lies on that plane, between the image of the point at infinity and the right camera center, within a χ^2 bound (one degree of freedom) of the right keypoint's octave. The best descriptor must pass a ratio test, matches are made one-to-one, and each pair is triangulated from both rays and accepted only if it reprojects within a two-degree-of-freedom χ^2 bound in both images. Triangulation search between keyframes uses the same bearing-to-plane distance.

Implementation. The projection formulas and their Jacobians are written once, in a single C++ header that depends only on Eigen and is shared by the Python and C++ cores. Optimization factors for g2o [50] and GTSAM [27] are implemented in C++ with analytic Jacobians, so no per-observation Python callbacks enter the optimization. Pinhole observations keep their original optimizer factors.

TUM-VI. The TUM-VI loader [90] supports two stereo modes. The *native* mode keeps the original fisheye images and builds two Kannala–Brandt cameras with the calibrated right-to-left transform. The *rectified* mode remaps the fisheye images once to a pinhole stereo pair, which then runs through the standard pinhole pipeline; this mode should be used with stereo depth estimators and dense reconstruction. Monocular fisheye tracking is also supported. Motion-capture poses are converted to the tracking camera frame and are never interpolated across gaps longer than a configurable threshold. Sect. 6.7 reports an initial comparison of the two modes.

Limitations. Stereo depth initialization and the legacy initialization and Sim(3) paths have not yet been validated for fields of view beyond 180°. Supporting a projection model is therefore not equivalent to supporting complete omnidirectional SLAM.

4.5 C++ Core for Sparse SLAM

The system provides a modular **sparse-SLAM core**, implemented in **both Python and C++** (with custom pybind11 bindings), allowing users to switch between high-performance/speed and high-flexibility modes. The C++ core reimplements the sparse SLAM initially implemented in Python, exposing core SLAM classes (frames, keyframes, map points, maps, cameras, optimizers, tracking, and local mapping) to Python via pybind11.

The C++ implementation follows a streamlined design where all core data resides in C++, with Python serving as an interface layer. C++ classes mirror their Python counterparts, maintaining identical interfaces and data

field names. The bindings support zero-copy data exchange (e.g., descriptors) and safe memory ownership across the Python/C++ boundary, leveraging automatic zero-copy sharing of NumPy array memory with C++ when possible.

The module is currently **under active development**.

The C++ core is selected by default (`USE_CPP_CORE = True` in `pyslam/config_parameters.py`); setting the flag to `False`, or launching with the environment variable `PYSLAM_USE_CPP=false`, selects the Python implementation. The two implementations are interoperable: a map saved by one can be loaded by the other. Two practical consequences follow from this design. When the C++ core is active, the Python implementation of the sparse core is bypassed, so changes to it have no effect at runtime. Conversely, changes to the C++ sources take effect only after the core is rebuilt with `./build_cpp_core.sh`.

The codebase is organized into the following categories:

- **Core SLAM Classes**: Main C++ implementations (`frame.cpp/h`, `keyframe.cpp/h`, `map_point.cpp/h`, `map.cpp/h`, `camera.cpp/h`, `camera_pose.cpp/h`, `optimizer_g2o.cpp/h`, `tracking_core.cpp/h`, `local_mapping_core.cpp/h`).
- **SLAM pipeline**: the `Slam`, `Tracking`, and `LocalMapping` classes (`slam.cpp/h`, `tracking.cpp/h`, `local_mapping.cpp/h`), which follow the control flow of their Python counterparts. Components that have not been ported, such as the feature tracker and loop closing, remain Python objects held by these classes.
- **Serialization**: JSON and NumPy serialization modules for data persistence and Python interop.
- **Python Bindings**: pybind11 module definitions that expose C++ classes to Python.
- **Type Casters**: pybind11 type casters for custom type conversions (OpenCV, Eigen, JSON, dictionary types).
- **Utilities**: Helper functions for geometry, features, descriptors, image processing, NumPy/Eigen operations, and serialization.
- **Tests**: C++ unit tests and Python integration tests (see Sect. 6.6).

The C++ core adopts a streamlined design philosophy:

- **All core data resides in C++**: Python serves purely as an interface layer.
- **Direct pybind11 exposure**: Python objects are lightweight views of underlying C++ objects.
- **Automatic zero-copy**: pybind11 automatically shares NumPy array memory with C++ when possible.
- **RAII-based ownership**: C++ smart pointers manage object lifetimes safely and efficiently.

4.6 3D Scene from Views

The diagram in Fig. 9 illustrates the architecture of the *Scene from Views* module, which provides a unified interface for feed-forward 3D scene reconstruction from multiple views. At its core, the `scene_from_views_factory` instantiates specific reconstruction models based on the selected `SceneFromViewsType`, such as `DUST3R`[112], `MAST3R`[51], `DEPTH_ANYTHING_V3`[56], `MVDUST3R`[99], `VGGT`[111], `VGGT_ROBUST`[38], and `FAST3R`[116].

Each type creates a corresponding implementation (e.g., `SceneFromViewsDust3r`, `SceneFromViewsMast3r`, `SceneFromViewsDepthAnythingV3`, `SceneFromViewsMvdust3r`, `SceneFromViewsVggt`, `SceneFromViewsVggtRobust`, `SceneFromViewsFast3r`), all inheriting from a common `SceneFromViewsBase`. This base class implements a unified three-step reconstruction pipeline: `preprocess_images()` prepares input images for the specific model, `infer()` runs model inference, and `postprocess_results()` converts raw model output to a standardized `SceneFromViewsResult` format containing merged point clouds, meshes, camera poses, and optional depth maps or intrinsics.

The module supports both pairwise models (`DUSt3R`[112], `MASt3R`[51]) that are feed-forward per pair and then run global alignment, as well as multi-view models (`DEPTH_ANYTHING_V3`[56], `MV-DUSt3R`[99], `VGGT`[111], `Robust VGGT`[38], `FAST3R`[116]) that process multiple views simultaneously in a single forward pass. This modular design enables flexible integration of diverse 3D reconstruction techniques while maintaining a consistent API across different model architectures.



Figure 9: *SceneFromViews* architecture.

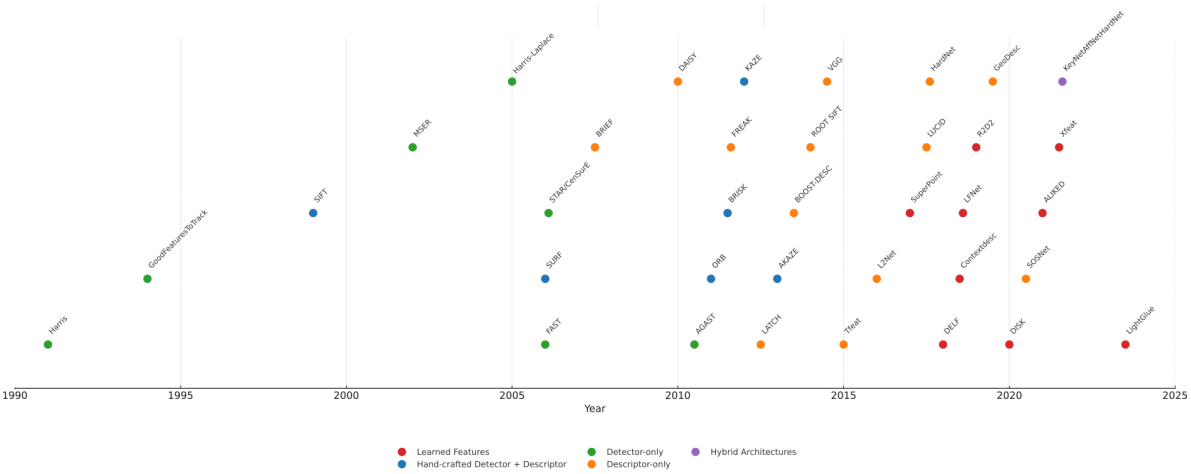


Figure 10: Timeline of some of the most famous local features for image matching, place recognition and SLAM.

5 Supported components and models

5.1 Supported local features

Fig. 10 shows a timeline with some of the most famous local features for image matching, place recognition and SLAM. At present time, pySLAM supports the following feature **detectors**:

- FAST [84]
- Good features to track [92]
- ORB [85]
- ORB2 (improvements of ORB-SLAM2[72] to ORB detector)
- SIFT [59]
- SURF [13]
- KAZE [3]
- AKAZE [4]

- BRISK [53]
- AGAST [63]
- MSER [65]
- StarDector/CenSurE [1]
- Harris-Laplace [67]
- SuperPoint[28]
- D2-Net [30]
- DELF [76]
- Contextdesc [61]
- LFNet [77]
- R2D2 [82]
- Key.Net [10]
- DISK [107]
- ALIKED [11]
- Xfeat [12]
- KeyNetAffNetHardNet (KeyNet detector [10] + AffNet + HardNet descriptor [69])

The following feature **descriptors** are supported:

- ORB [85]
- SIFT [59]
- ROOT SIFT [6]
- SURF [13]
- AKAZE [4]
- BRISK [53]
- FREAK [2]
- SuperPoint [28]
- Tfeat [8]
- BOOST-DESC [105]
- DAISY [104]
- LATCH [54]
- LUCID [60]
- VGG [93]
- Hardnet [69]
- GeoDesc [109]
- SOSNet [103]
- L2Net [102]
- Log-polar descriptor [31]
- D2-Net [30]
- DELF [76]
- Contextdesc [61]
- LFNet [77]
- R2D2 [82]
- BEBLID [96]

- DISK [107]
- ALIKED [11]
- Xfeat [12]
- KeyNetAffNetHardNet (KeyNet detector [10] + AffNet + HardNet descriptor [69])

For more information, refer to pyslam/local_features/feature_types.py file. Some of the local features consist of a *joint detector-descriptor*. You can start playing with the supported local features by taking a look at `examples/cv/example_feature_manager.py` and `main_feature_matching.py`.

In both the scripts `main_vo.py` and `main_slam.py`, you can create your preferred detector-descritor configuration and feed it to the function `feature_tracker_factory()`. Some ready-to-use configurations are already available in the file pyslam/local_features/feature_tracker_configs.py

The function `feature_tracker_factory()` can be found in the file `pyslam/local_features/feature_tracker.py`. Take a look at the file `pyslam/local_features/feature_manager.py` for further details.

N.B.: You just need a *single* python environment to be able to work with all the supported local features!

5.2 Supported matchers

- `BF`: Brute force matcher on descriptors (with KNN).
- FLANN [71]
- XFeat [12]
- LightGlue [58]
- LoFTR [97]
- MASt3R [52]

See the file `pyslam/local_features/feature_matcher.py` for further details.

5.3 Supported global descriptors and local descriptor aggregation methods

Local descriptor aggregation methods

- Bag of Words (BoW): DBoW2 [33], DBoW3 [33]. [paper]
- Vector of Locally Aggregated Descriptors: VLAD [41]. [paper]
- Incremental Bags of Binary Words (iBoW) via Online Binary Image Index: iBoW [34], OBIndex2 [34]. [paper]
- Hyperdimensional Computing: HDC [74]. [paper]

NOTE: *iBoW* and *OBIndex2* incrementally build a binary image index and do not need a prebuilt vocabulary. In the implemented classes, when needed, the input non-binary local descriptors are transparently transformed into binary descriptors.

Global descriptors

Also referred to as *holistic descriptors*:

- SAD [68]
- AlexNet [49]
- NetVLAD [5]
- HDC-DELF [87]
- CosPlace [15]
- EigenPlaces [16]
- MegaLoc [14]

Different loop closing methods are available. These combines the above aggregation methods and global descriptors. See the file pyslam/loop_closing/loop_detector_configs.py for further details.

5.4 Supported depth prediction models

Both monocular and stereo depth prediction models are available. SGBM algorithm has been included as a classic reference approach.

- SGBM[40]: Depth SGBM from OpenCV (Stereo, classic approach)
- Depth-Pro (Monocular) [17]
- DepthAnythingV2 (Monocular) [100]
- RAFT-Stereo (Stereo) [101]
- CREStereo (Stereo) [55]
- MASt3R (Monocular/Stereo) [52]
- MV-DUSt3R (Monocular/Stereo) [98]

5.5 Supported volumetric mapping methods

- A C++ template-based voxel grid implementation leverages parallel spatial hashing, supporting both direct voxel hashing and indirect voxel-block hashing strategies. Parallel execution is managed using TBB, and the design accommodates both simple and semantic voxels. Further information about the volumetric grid models is available here.
- TSDF with voxel block grid (parallel spatial hashing) [29]
- Incremental 3D Gaussian Splatting. See here and MonoGS for a description of its backend [66, 46].

5.6 Supported semantic segmentation methods

- DeepLabv3 [23]: from `torchvision`, pre-trained on COCO/VOC.
- Segformer [114]: from `transformers`, pre-trained on Cityscapes or ADE20k.
- CLIP [57]: from `f3rm` package for open-vocabulary support.
- EOV_SEG [75]: EOV-Seg: Efficient Open-Vocabulary Panoptic Segmentation.
- DETIC [117]: Detecting Twenty-thousand Classes using Image-level Supervision.
- ODISE [115]: Open-Vocabulary Panoptic Segmentation with Text-to-Image Diffusion Models.
- RFDETR [83]: A real-time transformer architecture for object detection and instance segmentation, built on a DINOv2 vision transformer backbone.
- YOLO [86]: One of the most influential and widely adopted series of models for real-time object detection, that attains an excellent balance between high accuracy and inference speed.

Supported models for feed-forward inference of 3D scenes from multiple images

- DUSt3R[112]: Geometric 3D Vision Made Easy
- MASt3R[52]: Grounding Image Matching in 3D with MASt3R
- Depth Anything V3[56]: Recovering the Visual Space from Any Views
- MVDust3r[98]: Multi-view DUSt3R variant
- VGGT[111]: Visual Geometry Grounded Transformer
- Robust VGGT[38]: Emergent Outlier View Rejection in Visual Geometry Grounded Transformers
- Fast3R[116]: Fast3R: Towards 3D Reconstruction of 1000+ Images in One Forward Pass

See here for further details about `SceneFromViews` architecture.

6 Experimental Evaluation

This section describes the tools pySLAM provides for evaluating SLAM configurations, reports results obtained with them on the TUM, EuRoC, and KITTI benchmarks, and presents the automated test suite and an initial validation of the camera models on TUM-VI.

6.1 Run a SLAM evaluation

The `main_slam_evaluation.py` script enables automated SLAM evaluation by executing `main_slam.py` across a collection of *datasets* and configuration *presets*. The main input to the script is an evaluation configuration file (e.g., `evaluation/configs/evaluation.json`) that specifies which datasets and presets to be used. For convenience, sample configurations for the datasets `TUM`, `EUROC` and `KITTI` datasets are already provided in the `evaluation/configs/` directory.

For each evaluation run, results are stored in a dedicated subfolder within the `results` directory, containing all the computed metrics. These metrics are then processed and compared. The final output is a report, available in PDF, LaTeX, and HTML formats, that includes comparison tables summarizing the *Absolute Trajectory Error* (ATE), the maximum deviation from the ground truth trajectory and other metrics.

Evaluation results are presented in the following subsections. For convenience, they are also available at this link. The evaluation uses commit *f7c9a13* with the following presets: `baseline`, `root_sift`, and `superpoint`, which correspond to local feature extractors `ORB2`, `root_SIFT`, and `SuperPoint`, respectively.

6.2 TUM Evaluation

Table 1: TUM: Table RMSE

Dataset	baseline	root_sift	superpoint
rgbd_dataset_freiburg1_desk	0.06634	0.05289	0.04815
rgbd_dataset_freiburg1_desk2	0.04645	0.06255	0.03868
rgbd_dataset_freiburg1_room	0.05885	0.07968	0.07199
rgbd_dataset_freiburg1_xyz	0.01196	0.01758	0.01726
rgbd_dataset_freiburg3_long_office_household	0.00921	0.00982	0.00952
rgbd_dataset_freiburg3_nostructure_texture_far	0.08397	0.11342	0.12118
rgbd_dataset_freiburg3_nostructure_texture_near_withloop	0.02245	0.05022	0.05037
Average	0.04275	0.05517	0.05102
Std Dev	0.0362	0.04531	0.04503
Best (Average) Preset	baseline		
Best (Average) Metric	0.04275		

Table 3: TUM: Table Max

Dataset	baseline	root_sift	superpoint
rgbd_dataset_freiburg1_desk	0.326	0.19361	0.22686
rgbd_dataset_freiburg1_desk2	0.44543	0.28865	0.38157
rgbd_dataset_freiburg1_room	0.14959	0.2838	0.17632
rgbd_dataset_freiburg1_xyz	0.06059	0.0677	0.0635
rgbd_dataset_freiburg3_long_office_household	0.03609	0.03543	0.04005
rgbd_dataset_freiburg3_nostructure_texture_far	0.29727	0.4361	0.70826
rgbd_dataset_freiburg3_nostructure_texture_near_withloop	0.09011	0.13774	0.11602
Average	0.20073	0.20615	0.24465
Std Dev	0.18661	0.18182	0.26334
Best (Average) Preset	baseline		
Best (Average) Metric	0.20073		

Table 5: TUM: Table Percent Lost

Dataset	baseline	root_sift	superpoint
rgbd_dataset_freiburg1_desk	1.364	1.502	1.674
rgbd_dataset_freiburg1_desk2	0.548	1.55	0.612
rgbd_dataset_freiburg1_room	0.074	0.148	0.088
rgbd_dataset_freiburg1_xyz	0.05	0.076	0.078
rgbd_dataset_freiburg3_long_office_household	0.0	0.0	0.0
rgbd_dataset_freiburg3_nostructure_texture_far	0.622	0.622	0.666
rgbd_dataset_freiburg3_nostructure_texture_near_withloop	0.024	0.036	0.036
Average	0.38314	0.562	0.45057
Std Dev	0.50438	0.82947	0.59847
Best (Average) Preset	baseline		
Best (Average) Metric	0.38314		

6.3 EUROC Evaluation

Table 7: EUROC: Table RMSE

Dataset	baseline	root_sift	superpoint
V101	0.08819	0.08816	0.08834
V102	0.06852	0.06609	0.06694
V201	0.07062	0.06409	0.09735
V202	0.62539	0.56925	0.53736
MH01	0.03775	0.03781	0.04172
MH02	0.04217	0.0456	0.04177
MH03	0.0512	0.04905	0.04882
MH04	0.0702	0.05907	0.0646
MH05	0.08177	0.06607	0.07131
Average	0.1262	0.11613	0.11758
Std Dev	0.19831	0.17517	0.15435
Best (Average) Preset	root_sift		
Best (Average) Metric	0.11613		

Table 9: EUROC: Table Max

Dataset	baseline	root_sift	superpoint
V101	0.16335	0.163	0.16208
V102	0.14545	0.12379	0.13136
V201	0.15864	0.18921	0.22163
V202	2.06607	2.52628	1.97847
MH01	0.09141	0.10259	0.21303
MH02	0.11922	0.1318	0.11196
MH03	0.19995	0.17533	0.14082
MH04	0.32021	0.27938	0.26722
MH05	0.24828	0.17474	0.18991
Average	0.39029	0.42957	0.37961
Std Dev	0.67128	0.75543	0.6093
Best (Average) Preset	superpoint		
Best (Average) Metric	0.37961		

Table 11: EUROC: Table Percent Lost

Dataset	baseline	root_sift	superpoint
V101	0.0	0.0	0.0
V102	0.0	0.0	0.0
V201	0.04	0.04	0.04
V202	0.146	0.138	0.112
MH01	0.05	0.074	0.08
MH02	0.018	0.026	0.012
MH03	0.046	0.03	0.032
MH04	0.0	0.01	0.0
MH05	0.07	0.016	0.034
Average	0.04111	0.03711	0.03444
Std Dev	0.05351	0.04829	0.04435
Best (Average) Preset	superpoint		
Best (Average) Metric	0.03444		

6.4 KITTI Evaluation

Table 13: KITTI: Table RMSE

Dataset	baseline	root_sift	superpoint
00	3.947	3.24849	3.93432
01	21.60014	21.86468	23.37292
02	6.1194	5.6707	10.14455
03	5.96451	5.92601	5.88895
04	1.74556	1.76121	1.65506
05	2.13915	2.11047	2.52263
06	2.74011	2.73981	2.6784
07	1.12358	1.12823	1.09496
08	3.99078	4.13702	4.14543
09	4.26068	4.35988	4.06244
10	2.22097	2.25127	2.27694
Average	5.07744	5.01798	5.61605
Std Dev	5.47265	5.63609	6.77
Best (Average) Preset	root_sift		
Best (Average) Metric	5.01798		

Table 15: KITTI: Table Max

Dataset	baseline	root_sift	superpoint
00	6.82983	5.8663	7.04067
01	33.32903	33.85867	36.62253
02	13.97623	9.47974	21.05188
03	10.16642	10.16978	10.06014
04	3.09135	3.0576	2.87271
05	3.81484	3.77687	4.9547
06	6.53758	5.14239	5.11673
07	1.84418	1.82355	1.8655
08	10.809	11.23635	10.75959
09	8.53888	8.70237	8.31166
10	4.13901	4.37849	4.27905
Average	9.37058	8.86292	10.26683
Std Dev	8.62548	8.62145	12.23529
Best (Average) Preset	root_sift		
Best (Average) Metric	8.86292		

Table 17: KITTI: Table Percent Lost

Dataset	baseline	root_sift	superpoint
00	0.0	0.038	0.012
01	0.0	0.0	0.0
02	0.012	0.004	0.004
03	0.0	0.0	0.0
04	0.0	0.0	0.0
05	0.0	0.0	0.028
06	0.108	0.018	0.0
07	0.0	0.0	0.0
08	0.0	0.0	0.0
09	0.0	0.0	0.0
10	0.0	0.0	0.0
Average	0.01091	0.00545	0.004
Std Dev	0.07244	0.02373	0.0196
Best (Average) Preset	superpoint		
Best (Average) Metric	0.004		

6.5 pySLAM performances and comparative evaluations

For a comparative evaluation of the "*online*" trajectory estimated by pySLAM versus the "*final*" trajectory estimated by ORB-SLAM3, check out this nice notebook. For more details about "online" and "final" trajectories, refer to Section A.7.

Note: Unlike ORB-SLAM3, which only saves the final pose estimates (recorded after the entire dataset has been processed), pySLAM saves both online and final pose estimates. For details on how to save trajectories in pySLAM, refer to this section. When you click the `Draw Ground Truth` button in the GUI (see here), you can visualize the *Absolute Trajectory Error* (ATE or *RMSE*) history and evaluate both online and final errors up to the current time.

6.6 Automated Testing

Reproducibility also depends on detecting regressions as the framework evolves. The repository therefore separates the automated test suite (folder `tests`) from the usage examples (folder `examples`). The suite comprises Python unit tests run with `pytest`, C++ unit tests of the sparse core, Python tests of the C++ core and its bindings, and headless checks of the C++ extension modules. A single launcher, `tests/run_all.sh`, runs all of them in sequence and reports the failing steps. Several tests reproduce past defects, such as crashes in local mapping and relocalization or inconsistent loop-detector database indices, so that they cannot reappear unnoticed.

Changes that affect the camera models are additionally checked end to end. A replay harness processes a dataset sequence deterministically (ORB2 features, synchronous local mapping, and no loop closure unless requested) and records tracking states, per-frame timings, keyframe and map-point counts, and the absolute trajectory error, with Sim(3) alignment for monocular runs and SE(3) alignment for metric sensors. A comparison script replays the same sequences with a baseline checkout and with the current one, alternating their order, and fails if the median paired frame-time ratio exceeds 1.20, if any pair loses more than two percentage points of tracking or ground-truth coverage, or if the trajectory error grows by more than the larger of 20% and 0.02 m.

6.7 Camera Models on TUM-VI

As an initial check of the camera models of Sect. 4.4, the first 300 frames of the TUM-VI `room1` sequence (512×512) were replayed in the rectified and native stereo modes, using the C++ core, g2o, ORB2 with 1200 features, and synchronous local mapping. Table 19 summarizes the results. Both modes tracked all frames after initialization. The rectified mode, which runs the established pinhole pipeline, was faster and more accurate on this short segment; the native mode pays for bearing-based stereo matching and model-aware projection and optimization.

Table 19: TUM-VI `room1`, first 300 frames: rectified versus native fisheye stereo. Timings include tracking and synchronous local mapping.

Measurement	Rectified	Native
Tracking states	1 initializing, 299 OK	1 initializing, 299 OK
Trajectory / mocap associations	299 / 299	299 / 299
SE(3)-aligned translation RMSE	0.0061 m	0.0127 m
Mean tracking + local mapping	81.5 ms/frame	238.1 ms/frame
Median tracking + local mapping	38.4 ms/frame	188.0 ms/frame
Mean image loading and preprocessing	14.5 ms/pair	9.7 ms/pair
Final keyframes / map points	48 / 2141	54 / 2417

These are exploratory measurements on a short segment, not a controlled benchmark: timings include warm-up and logging, and full-sequence drift, loop closing, the remaining TUM-VI sequences, and the Python and GTSAM paths on real data remain to be evaluated.

7 Conclusions and Future Work

This document presented pySLAM, an open-source framework for visual SLAM that combines a sparse SLAM core, available in both Python and C++, with interchangeable local features, loop detectors, depth estimators, semantic segmentation models, volumetric integrators, and feed-forward multi-view reconstruction models. A unified dataset interface, automated evaluation scripts, and a regression test suite make it possible to compare these components within a complete pipeline and across more than fifteen dataset types. The framework is intended as a research and teaching platform rather than a real-time system.

Several directions remain open. Inertial and LiDAR measurements are not yet integrated, and the tracker does not include direct methods. Object-level mapping is under development. For the camera models, wide fields of view beyond 180° still need to be validated in initialization and Sim(3) estimation, and the native fisheye pipeline needs a full-sequence evaluation on TUM-VI and other fisheye datasets.

Acknowledgments

A list of the frameworks that inspired or has been integrated into pySLAM are available here. Many thanks to their Authors for their great work.

References

[1] Motilal Agrawal, Kurt Konolige, and Morten Rufus Blas. Censure: Center surround extremas for realtime feature detection and matching. In David Forsyth, Philip Torr, and Andrew Zisserman, editors, *Computer Vision – ECCV 2008*, pages 102–115, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.

[2] Alexandre Alahi, Raphael Ortiz, and Pierre Vandergheynst. Freak: Fast retina keypoint. In *2012 IEEE conference on computer vision and pattern recognition*, pages 510–517. Ieee, 2012.

[3] Pablo F Alcantarilla, Adrien Bartoli, and Andrew J Davison. Kaze features. *European conference on computer vision*, pages 214–227, 2012.

[4] Pablo F Alcantarilla, Jesús Nuevo, and Adrien Bartoli. Fast explicit diffusion for accelerated features in nonlinear scale spaces. *IEEE transactions on pattern analysis and machine intelligence*, 34(7):1281–1298, 2013.

[5] Relja Arandjelovic, Petr Gronat, Akihiko Torii, Tomas Pajdla, and Josef Sivic. Netvlad: Cnn architecture for weakly supervised place recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5297–5307, 2016.

[6] Relja Arandjelović and Andrew Zisserman. Three things everyone should know to improve object retrieval. In *2012 IEEE conference on computer vision and pattern recognition*, pages 2911–2918. IEEE, 2012.

[7] Dejan Azinović, Ricardo Martin-Brualla, Dan B Goldman, Matthias Nießner, and Justus Thies. Neural rgb-d surface reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6290–6301, June 2022.

[8] Vassileios Balntas, Edgar Riba, Daniel Ponsa, and Krystian Mikolajczyk. Learning local feature descriptors with triplets and shallow convolutional neural networks. In *Bmvc*, volume 1, page 3, 2016.

[9] Yanqi Bao, Tianyu Ding, Jing Huo, Yaoli Liu, Yuxin Li, Wenbin Li, Yang Gao, and Jiebo Luo. 3d gaussian splatting: Survey, technologies, challenges, and opportunities. *IEEE Transactions on Circuits and Systems for Video Technology*, 2025.

[10] Axel Barroso-Laguna, Edgar Riba, Daniel Ponsa, and Krystian Mikolajczyk. Key.net: Keypoint detection by handcrafted and learned cnn filters. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5836–5844, 2020.

[11] Axel Barroso-Laguna, Edgar Riba, Daniel Ponsa, and Krystian Mikolajczyk. Aliked: A lightweight keypoint detector and descriptor. *arXiv preprint arXiv:2304.03608*, 2023.

[12] Axel Barroso-Laguna, Edgar Riba, Daniel Ponsa, and Krystian Mikolajczyk. Xfeat: A new feature detector and descriptor. *arXiv preprint arXiv:2404.19174*, 2024.

[13] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: Speeded up robust features. *European conference on computer vision*, pages 404–417, 2006.

[14] Gabriele Berton and Carlo Masone. Megaloc: One retrieval to place them all. *arXiv preprint arXiv:2502.17237*, 2025.

[15] Gabriele Berton, Carlo Masone, and Barbara Caputo. Cosplace: Efficient place recognition with cosine similarity. *arXiv preprint arXiv:2304.03608*, 2023.

[16] Gabriele Berton, Carlo Masone, and Barbara Caputo. Eigenplaces: Learning place recognition with eigenvectors. *arXiv preprint arXiv:2404.19174*, 2023.

[17] Aleksei Bochkovskii, Amaël Delaunoy, Hugo Germain, Marcel Santos, Yichao Zhou, Stephan R Richter, and Vladlen Koltun. Depth pro: Sharp monocular metric depth in less than a second. *arXiv preprint arXiv:2410.02073*, 2024.

[18] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[19] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W Achtelik, and Roland Siegwart. The euroc micro aerial vehicle datasets. *The International Journal of Robotics Research*, 2016.

[20] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, José Neira, Ian Reid, and John J Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on robotics*, 32(6):1309–1332, 2017.

[21] Carlos Campos, Richard Elvira, Juan J. Gómez Rodríguez, José M. M. Montiel, and Juan D. Tardós. Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.

[22] Kaiqi Chen, Junhao Xiao, Jialing Liu, Qiyi Tong, Heng Zhang, Ruyu Liu, Jianhua Zhang, Arash Ajoudani, and Shengyong Chen. Semantic visual simultaneous localization and mapping: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[23] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017.

[24] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. *CoRR*, abs/1604.01685, 2016.

[25] Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. In *Proc. Computer Vision and Pattern Recognition (CVPR), IEEE*, 2017.

[26] Andrew J Davison. Futuremapping: The computational structure of spatial ai systems. *arXiv preprint arXiv:1803.11288*, 2018.

[27] Frank Dellaert and Michael Kaess. *Factor Graphs for Robot Perception*. Foundations and Trends in Robotics, Vol. 6, 2017.

[28] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. Superpoint: Self-supervised interest point detection and description. In *CVPR Deep Learning for Visual SLAM Workshop*, 2018.

[29] Wei Dong, Yixing Lao, Michael Kaess, and Vladlen Koltun. Ash: A modern framework for parallel spatial hashing in 3d perception. *IEEE transactions on pattern analysis and machine intelligence*, 45(5):5417–5435, 2022.

[30] Mihai Dusmanu, Ignacio Rocco, Tomas Pajdla, Marc Pollefeys, Josef Sivic, Akihiko Torii, and Torsten Sattler. D2-net: A trainable cnn for joint description and detection of local features. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8092–8101, 2019.

[31] Patrick Ebel, Anastasiia Mishchuk, Kwang Moo Yi, Pascal Fua, and Eduard Trulls. Beyond Cartesian Representations for Local Descriptors. 2019.

[32] Luigi Freda. Plvs: A slam system with points, lines, volumetric mapping, and 3d incremental segmentation. *arXiv preprint arXiv:2309.10896*, 2023.

[33] Dorian Galvez-Lopez and Juan D Tardos. Bags of binary words for fast place recognition in image sequences. *IEEE Transactions on Robotics*, 28(5):1188–1197, 2012.

[34] Emilio Garcia-Fidalgo and Alberto Ortiz. ibow-lcd: An appearance-based loop closure detection approach using incremental bags of binary words. *IEEE Robotics and Automation Letters*, 3(4):3051–3057, 2018.

[35] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.

[36] Bruno Georgevich Ferreira, Armando Jorge Miranda de Sousa, and Luis Reis. Semantic mapping for robotics: Survey, trends and challenges. *Trends and Challenges*.

[37] Ben Glocker, Shahram Izadi, Jamie Shotton, and Antonio Criminisi. Real-time rgb-d camera relocalization. In *2013 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pages 173–179, 2013.

[38] Jisang Han, Sunghwan Hong, Jaewoo Jung, Wooseok Jang, Honggyu An, Qianqian Wang, Seungryong Kim, and Chen Feng. Emergent outlier view rejection in visual geometry grounded transformers. *arXiv preprint arXiv:2512.04012*, 2025.

[39] A. Handa, T. Whelan, J.B. McDonald, and A.J. Davison. A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM. In *IEEE Intl. Conf. on Robotics and Automation, ICRA*, Hong Kong, China, May 2014.

[40] Heiko Hirschmuller. Stereo processing by semiglobal matching and mutual information. *IEEE Transactions on pattern analysis and machine intelligence*, 30(2):328–341, 2007.

[41] Hervé Jégou, Matthijs Douze, Cordelia Schmid, and Patrick Pérez. Aggregating local descriptors into a compact image representation. In *2010 IEEE computer society conference on computer vision and pattern recognition*, pages 3304–3311. IEEE, 2010.

[42] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Yolov8, 2023.

[43] Glenn Jocher et al. Yolov5, 2020.

[44] Khushbu Joshi and Manish I Patel. Recent advances in local feature detector and descriptor: a literature survey. *International Journal of Multimedia Information Retrieval*, 9(4):231–247, 2020.

[45] Juho Kannala and Sami S. Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(8):1335–1340, 2006.

[46] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.

[47] Hannah Kim, Nathan Hughes, Yun Chang, and Luca Carlone. Clio: Real-time task-driven open-set 3d scene graphs. *arXiv preprint arXiv:2404.13696*, 2024.

[48] Georg Klein and David Murray. Parallel tracking and mapping for small ar workspaces. In *2007 6th IEEE and ACM International Symposium on Mixed and Augmented Reality*, pages 225–234, 2007.

[49] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[50] Rainer Kümmerle, Giorgio Grisetti, Hauke Strasdat, Kurt Konolige, and Wolfram Burgard. g 2 o: A general framework for graph optimization. In *2011 IEEE international conference on robotics and automation*, pages 3607–3613. IEEE, 2011.

[51] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding image matching in 3d with mast3r. In *European Conference on Computer Vision*, pages 71–91. Springer, 2024.

[52] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding image matching in 3d with mast3r, 2024.

[53] Stefan Leutenegger, Margarita Chli, and Roland Y Siegwart. Brisk: Binary robust invariant scalable keypoints. *2011 International conference on computer vision*, pages 2548–2555, 2011.

[54] Gil Levi, Tal Hassner, and Ronen Basri. The latch descriptor: Local binary patterns for image matching. *IEEE transactions on pattern analysis and machine intelligence*, 38(8):1622–1634, 2016.

[55] Zhengfa Li, Yuhua Liu, Tianwei Shen, Shuaicheng Chen, Lu Fang, and Long Quan. Crestereo: Cross-scale cost aggregation for stereo matching. *arXiv preprint arXiv:2203.11483*, 2022.

[56] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.

[57] Yuqi Lin, Minghao Chen, Wenxiao Wang, Boxi Wu, Ke Li, Binbin Lin, Haifeng Liu, and Xiaofei He. Clip is also an efficient segmenter: A text-driven approach for weakly supervised semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15305–15314, 2023.

[58] Philipp Lindenberger, Paul-Edouard Sarlin, and Marc Pollefeys. Lightglue: Local feature matching at light speed. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 17627–17638, 2023.

[59] David G Lowe. Object recognition from local scale-invariant features. *Proceedings of the seventh IEEE international conference on computer vision*, 2:1150–1157, 1999.

[60] Zixin Luo, Tianwei Shen, Lei Zhou, Siyu Zhu, Runze Zhang, Yao Yao, Tian Fang, and Long Quan. Lucid: A compact local image descriptor for real-time applications. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 1–9, 2015.

[61] Zixin Luo, Lei Zhou, Xiang Bai, Alan Yuille, and Jimmy Ren. Contextdesc: Local descriptor augmentation with cross-modality context. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2527–2536, 2020.

[62] Jiayi Ma, Xingyu Jiang, Aoxiang Fan, Junjun Jiang, and Junchi Yan. Image matching from handcrafted to deep features: A survey. *International Journal of Computer Vision*, 129(1):23–79, 2021.

[63] Elmar Mair, Gregory D Hager, Darius Burschka, Michael Suppa, and Gerhard Hirzinger. Adaptive and generic corner detection based on the accelerated segment test. In *European conference on Computer vision*, pages 183–196. Springer, 2010.

[64] Satya Mallick. Camera calibration using opencv, 2016.

[65] Jiri Matas, Ondrej Chum, Martin Urban, and Tomas Pajdla. Robust wide-baseline stereo from maximally stable extremal regions. *Proceedings of the British Machine Vision Conference*, 1(502):384–393, 2002.

[66] H Matsuki, R Murai, PH Kelly, and AJ Davison. Gaussian splatting slam. arxiv. *arXiv preprint arXiv:2312.06741*, 2023.

[67] Krystian Mikolajczyk and Cordelia Schmid. Scale & affine invariant interest point detectors. *International Journal of Computer Vision*, 60(1):63–86, 2004.

[68] Michael J Milford and Gordon F Wyeth. Seqslam: Visual route-based navigation for sunny summer days and stormy winter nights. In *2012 IEEE international conference on robotics and automation*, pages 1643–1649. IEEE, 2012.

[69] Anastasiia Mishchuk, Dmytro Mishkin, Filip Radenovic, and Jiri Matas. Working hard to know your neighbor's margins: Local descriptor learning loss. *Advances in neural information processing systems*, 30, 2017.

[70] David Morilla-Cabello and Eduardo Montijano. Semantic pyslam: Unifying semantic mapping approaches under the same framework. *RSS 2025 Workshop, Unifying Visual SLAM*, 2025.

[71] Marius Muja and David G Lowe. Fast approximate nearest neighbors with automatic algorithm configuration. *VISAPP (1)*, 2(331-340):2, 2009.

[72] Raul Mur-Artal and Juan D. Tardos. Orb-slam2: An open-source slam system for monocular, stereo and rgb-d cameras, 2017.

[73] Pushmeet Kohli Nathan Silberman, Derek Hoiem and Rob Fergus. Indoor segmentation and support inference from rgbd images. In *ECCV*, 2012.

[74] Peer Neubert and Peter Protzel. Hyperdimensional computing as a framework for systematic aggregation of image descriptors. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9067–9076, 2021.

[75] Hongwei Niu, Jie Hu, Jianghang Lin, Guannan Jiang, and Shengchuan Zhang. Eov-seg: Efficient open-vocabulary panoptic segmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 6254–6262, 2025.

[76] Hyeonwoo Noh, Andre Araujo, Joonseok Sim, Tobias Weyand, and Bohyung Han. Large-scale image retrieval with attentive deep local features. *Proceedings of the IEEE international conference on computer vision*, pages 3456–3465, 2017.

[77] Yoshitaka Ono, Eduard Trulls, Pascal Fua, and Kwang Moo Yi. Lf-net: Learning local features from images. *Advances in neural information processing systems*, 31, 2018.

[78] OpenCV. Camera calibration, 2021.

[79] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[80] Joseph Redmon and Ali Farhadi. Yolo9000: Better, faster, stronger. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7263–7271, 2017.

[81] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[82] Jerome Revaud, Philippe Weinzaepfel, Cedric R De Souza, Nicolas Pion, Gabriela Csurka, Yohann Cabon, and Martin Humenberger. R2d2: Repeatable and reliable detector and descriptor. *Advances in neural information processing systems*, 32, 2019.

[83] Isaac Robinson, Peter Robicheaux, Matvei Popov, Deva Ramanan, and Neehar Peri. Rf-detr: Neural architecture search for real-time detection transformers, 2025.

[84] Edward Rosten and Tom Drummond. Machine learning for high-speed corner detection. *European conference on computer vision*, pages 430–443, 2006.

[85] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. Orb: An efficient alternative to sift or surf. *2011 International conference on computer vision*, pages 2564–2571, 2011.

[86] Ranjan Sapkota and Manoj Karkee. Ultralytics yolo evolution: An overview of yolo26, yolo11, yolov8 and yolov5 object detectors. *arXiv preprint arXiv:2510.09653*, 2025.

[87] Kenny Schlegel, Peer Neubert, and Peter Protzel. A comparison of vector symbolic architectures. *Artificial Intelligence Review*, 55(6):4523–4555, 2022.

[88] Lukas Schmid, Marcus Abate, Yun Chang, and Luca Carlone. Khronos: A unified approach for spatio-temporal metric-semantic slam in dynamic environments. *arXiv preprint arXiv:2402.13817*, 2024.

[89] Fabian Schmidt, Julian Daubermann, Marcel Mitschke, Constantin Blessing, Stephan Meyer, Markus Enzweiler, and Abhinav Valada. Rover: A multiseason dataset for visual slam. *IEEE Transactions on Robotics*, 41:4005–4022, 2025.

[90] David Schubert, Thore Goll, Nikolaus Demmel, Vladyslav Usenko, Jörg Stückler, and Daniel Cremers. The tum vi benchmark for evaluating visual-inertial odometry. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1680–1687. IEEE, 2018.

[91] Stefan Schubert, Peer Neubert, Sourav Garg, Michael Milford, and Tobias Fischer. Visual place recognition: A tutorial [tutorial]. *IEEE Robotics & Automation Magazine*, 31(3):139–153, 2023.

[92] Jianbo Shi and Carlo Tomasi. Good features to track. *1994 Proceedings of IEEE conference on computer vision and pattern recognition*, pages 593–600, 1994.

[93] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Learning local feature descriptors using convex optimisation. *IEEE transactions on pattern analysis and machine intelligence*, 36(8):1573–1585, 2014.

[94] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J. Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, Anton Clarkson, Mingfei Yan, Brian Budge, Yajie Yan, Xiaqing Pan, June Yon, Yuyang Zou, Kimberly Leon, Nigel Carter, Jesus Briales, Tyler Gillingham, Elias Mueggler, Luis Pesqueira, Manolis Savva, Dhruv Batra, Hauke M. Strasdat, Renzo De Nardi, Michael Goesele, Steven Lovegrove, and Richard Newcombe. The Replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019.

[95] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of rgb-d slam systems. In *Proc. of the International Conference on Intelligent Robot Systems (IROS)*, Oct. 2012.

[96] Iago Suárez, Ghesn Sfeir, José M Buenaposada, and Luis Baumela. Beblid: Boosted efficient binary local image descriptor. *Pattern recognition letters*, 133:366–372, 2020.

[97] Jiaming Sun, Zehong Shen, Yuang Wang, Hujun Bao, and Xiaowei Zhou. Loftr: Detector-free local feature matching with transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8922–8931, 2021.

[98] Zhenggang Tang, Yuchen Fan, Dilin Wang, Hongyu Xu, Rakesh Ranjan, Alexander Schwing, and Zhicheng Yan. Mv-dust3r+: Single-stage scene reconstruction from sparse views in 2 seconds, 2024.

[99] Zhenggang Tang, Yuchen Fan, Dilin Wang, Hongyu Xu, Rakesh Ranjan, Alexander Schwing, and Zhicheng Yan. Mv-dust3r+: Single-stage scene reconstruction from sparse views in 2 seconds. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5283–5293, 2025.

[100] DepthAnything Team. Depthanythingv2: A monocular depth prediction model. *arXiv preprint arXiv:2406.09414*, 2024.

[101] Zachary Teed and Jia Deng. Raft-stereo: Recurrent all-pairs field transforms for stereo matching. *arXiv preprint arXiv:2109.07547*, 2021.

[102] Yurun Tian, Bin Fan, and Fuchao Wu. L2-net: Deep learning of discriminative patch descriptor in euclidean space. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 661–669, 2017.

[103] Yurun Tian, Xin Yu, Bin Fan, Fuchao Wu, Huub Heijnen, and Vassileios Balntas. Sosnet: Second order similarity regularization for local descriptor learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11016–11025, 2019.

[104] Engin Tola, Vincent Lepetit, and Pascal Fua. Daisy: An efficient dense descriptor applied to wide-baseline stereo. *IEEE transactions on pattern analysis and machine intelligence*, 32(5):815–830, 2010.

[105] Tomasz Trzcinski, Marios Christoudias, Pascal Fua, and Vincent Lepetit. Boosting binary keypoint descriptors. *2013 IEEE Conference on Computer Vision and Pattern Recognition*, pages 2874–2881, 2013.

[106] Tinne Tuytelaars, Krystian Mikolajczyk, et al. Local invariant feature detectors: a survey. *Foundations and trends® in computer graphics and vision*, 3(3):177–280, 2008.

[107] Maciej Tyszkiewicz, Pascal Fua, and Eduard Trulls. Disk: Learning local features with policy gradient. *Advances in neural information processing systems*, 33:14254–14265, 2020.

[108] Vladyslav Usenko, Nikolaus Demmel, and Daniel Cremers. The double sphere camera model. In *2018 International Conference on 3D Vision (3DV)*, pages 552–560. IEEE, 2018.

[109] Yannick Verdie, Kwang Moo Yi, Pascal Fua, and Vincent Lepetit. Tilde: A temporally invariant learned detector. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5279–5288, 2015.

[110] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. *arXiv preprint arXiv:2207.02696*, 2022.

[111] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5294–5306, 2025.

[112] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jérôme Revaud. Dust3r: Geometric 3d vision made easy. 2024 ieee. In *CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20697–20709, 2023.

[113] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. Tartanair: A dataset to push the limits of visual slam. 2020.

[114] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M Alvarez, and Ping Luo. Segformer: Simple and efficient design for semantic segmentation with transformers. *Advances in neural information processing systems*, 34:12077–12090, 2021.

[115] Jiarui Xu, Sifei Liu, Arash Vahdat, Wonmin Byeon, Xiaolong Wang, and Shalini De Mello. Open-vocabulary panoptic segmentation with text-to-image diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2955–2966, 2023.

[116] Jianing Yang, Alexander Sax, Kevin J Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. Fast3r: Towards 3d reconstruction of 1000+ images in one forward pass. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21924–21935, 2025.

[117] Xingyi Zhou, Rohit Girdhar, Armand Joulin, Philipp Krähenbühl, and Ishan Misra. Detecting twenty-thousand classes using image-level supervision. In *European conference on computer vision*, pages 350–368. Springer, 2022.

[118] Michael Zollhöfer, Martin Martinek, Jörg Stückler, Reinhard Klein, Gordon Wetzstein, and Matthias Nießner. State of the art on 3d reconstruction with rgb-d cameras. *Computer Graphics Forum*, 37(2):625–652, 2018.

A Usage

Open a new terminal and start experimenting with the scripts. In each new terminal you are supposed to start with this command:

```
#pixi shell           # If you use pixi, this activates the pyslam environment.
 . pyenv-activate.sh   #  Activate pyslam python virtual environment. This is only needed once in a new terminal.
```

If you are using `pixi` then just run `pixi shell` to activate the `pyslam` environment. The file config.yaml can be used as a unique entry-point to configure the system and its global configuration parameters contained in pyslam/config_parameters.py.

- The **C++ sparse-SLAM core** is enabled by default: the flag `USE_CPP_CORE` in the file `pyslam/config_parameters.py` is `True`. To use the Python core, set the flag to `False` or launch with `PYSLAM_USE_CPP=false` (see Sect. 4.5).
- Further information on how to configure pySLAM is provided here.

A.1 Visual odometry

The basic **Visual Odometry** (VO) can be run with the following commands:

```
#pixi shell           # If you use pixi, this activates the pyslam environment.
$ . pyenv-activate.sh   #  Activate pyslam python virtual environment. This is only needed once in a new terminal.
$ ./main_vo.py
```

By default, the script processes a KITTI video (available in the folder `data/videos`) by using its corresponding camera calibration file (available in the folder `settings`), and its groundtruth (available in the same `data/videos` folder). If matplotlib windows are used, you can stop `main_vo.py` by clicking on one of them and pressing the key 'Q'. As explained above, this very *basic* script `main_vo.py` **strictly requires a ground truth**. Now, with RGBD datasets, you can also test the **RGBD odometry** with the classes `VisualOdometryRgbd` or `VisualOdometryRgbdTensor` (ground truth is not required here).

A.2 Full SLAM

Similarly, you can test the **full SLAM** by running `main_slam.py`:

```
#pixi shell           # If you use pixi, this activates the pyslam environment.
$ . pyenv-activate.sh   #  Activate pyslam python virtual environment. This is only needed once in a new terminal.
$ ./main_slam.py
```

This will process the same default KITTI video (available in the folder `data/videos`) by using its corresponding camera calibration file (available in the folder `settings`). You can stop it by clicking on one of the opened windows and pressing the key 'Q' or closing the 3D pangolin GUI.

A.3 Selecting a dataset and different configuration parameters

The file config.yaml can be used as a unique entry-point to configure the system, the target dataset and its global configuration parameters set in pyslam/config_parameters.py. To process a different **dataset** with both VO and SLAM scripts, you need to update the file config.yaml:

- Select your dataset `type` in the section `DATASET` (further details in the section *Datasets* below for further details). This identifies a corresponding dataset section (e.g. `KITTI_DATASET`, `TUM_DATASET`, etc).
- Select the `sensor_type` (`mono`, `stereo`, `rgbd`) in the chosen dataset section.
- Select the camera `settings` file in the dataset section (further details in the section *Camera Settings* below).
- Set the `groundtruth_file` accordingly. Further details in the section *Datasets* below (see also the files `pyslam/io/ground_truth.py` and `pyslam/io/convert_groundtruth_to_simple.py`).

You can use the section `GLOBAL_PARAMETERS` of the file config.yaml to override the global configuration parameters set in pyslam/config_parameters.py. This is particularly useful when running a SLAM evaluation.

A.4 Feature tracking

If you just want to test the basic feature tracking capabilities (*feature detector* + *feature descriptor* + *feature matcher*) and get a taste of the different available local features, run

```
#pixi shell           # If you use pixi, this activates the pyslam environment.
$ . pyenv-activate.sh    #  Activate pyslam python virtual environment. This is only needed once in a new terminal.
$./main_feature_matching.py
```

In any of the above scripts, you can choose any detector/descriptor among *ORB*, *SIFT*, *SURF*, *BRISK*, *AKAZE*, *SuperPoint*, etc. (see the section *Supported Local Features* below for further information).

Some basic examples are available in the subfolder `examples/cv`. In particular, as for feature detection/description, you may want to take a look at examples/cv/example_feature_manager.py too.

A.5 Loop closing

Many loop closing methods are available, combining different aggregation methods and global descriptors.

While running full SLAM, loop closing is enabled by default and can be disabled by setting `kUseLoopClosing=False` in `pyslam/config_parameters.py`. Different configuration options `LoopDetectorConfigs` can be found in pyslam/loop_closing/loop_detector_configs.py: Code comments provide additional useful details.

One can start experimenting with loop closing methods by using the examples in `examples/loopclosing`. The example examples/loopclosing/example_loop_detector.py is the recommended entry point.

Vocabulary management

`DBoW2`, `DBoW3`, and `VLAD` require **pre-trained vocabularies**. ORB-based vocabularies are automatically downloaded into the `data` folder (see pyslam/loop_closing/loop_detector_configs.py).

To create a new vocabulary, follow these steps:

1. **Generate an array of descriptors**: Use the script `examples/loopclosing/example_gen_des_array_from_imgs.py` to generate the array of descriptors that will be used to train the new vocabulary. Select your desired descriptor type via the tracker configuration.
2. **DBOW vocabulary generation**: Train your target DBOW vocabulary by using the script `examples/loopclosing/example_gen_dbow_voc_from_des_array.py`.
3. **VLAD vocabulary generation**: Train your target VLAD "vocabulary" by using the script `examples/loopclosing/example_gen_vlad_voc_from_des_array.py`.

Once you have trained the vocabulary, you can add it in pyslam/loop_closing/loop_detector_vocabulary.py and correspondingly create a new loop detector configuration in pyslam/loop_closing/loop_detector_configs.py that uses it.

Vocabulary-free loop closing

Most methods do not require pre-trained vocabularies. Specifically:

- `iBoW` and `OBindex2`: These methods incrementally build bags of binary words and, if needed, convert (front-end) non-binary descriptors into binary ones.
- Others: Methods like `HDC_DELF`, `SAD`, `AlexNet`, `NetVLAD`, `CosPlace`, and `EigenPlaces`, and `Megaloc` directly extract their specific **global descriptors** and process them using dedicated aggregators, independently from the used front-end descriptors.

As mentioned above, only `DBoW2`, `DBoW3`, and `VLAD` require pre-trained vocabularies.

Verify your loop detection configuration and verify vocabulary compability

Loop detection method based on a pre-trained vocabulary

When selecting a **loop detection method based on a pre-trained vocabulary** (such as `DBoW2`, `DBoW3`, and `VLAD`), ensure the following:

1. The back-end and the front-end are using the same descriptor type (this is also automatically checked for consistency) or their descriptor managers are independent (see further details in the configuration options `LoopDetectorConfigs` available in pyslam/loop_closing/loop_detector_configs.py).
2. A corresponding pre-trained vocubulary is available. For more details, refer to the vocabulary management section.

Missing vocabulary for the selected front-end descriptor type

If you lack a compatible vocabulary for the selected front-end descriptor type, you can follow one of these options:
1. Create and load the vocabulary (refer to the vocabulary management section).
2. Choose an `*_INDEPENDENT` loop detector method, which works with an independent local_feature_manager.
3. Select a vocabulary-free loop closing method.

See the file pyslam/loop_closing/loop_detector_configs.py for further details.

A.6 Volumetric reconstruction

Dense reconstruction while running SLAM

The SLAM back-end hosts a volumetric reconstruction pipeline. This is disabled by default. You can enable it by setting `kDoVolumetricIntegration=True` and selecting your preferred method `kVolumetricIntegrationType` in `pyslam/config_parameters.py`. At present, the following methods are available:

- `VOXEL_GRID`, `VOXEL_SEMANTIC_GRID`, `VOXEL_SEMANTIC_PROBABILISTIC_GRID`: Voxel grid implementations that leverage parallel spatial hashing, supporting both direct voxel hashing and indirect voxel-block hashing strategies. Parallel execution is managed using TBB, and the design accommodates both simple and semantic voxels.
- `TSDF` (Truncated Signed Distance Function): It is able to return in output either a pointcloud or a mesh.
- `GAUSSIAN_SPLATTING` (Incremental Gaussian splatting)pyslam/dense/volumetric_integrator_factory.py).

Note that you need CUDA in order to run `GAUSSIAN_SPLATTING` method.

The volumetric reconstruction pipeline works with:

- RGBD datasets
- When a depth estimator is used
  - in the back-end with STEREO datasets (you can't use depth prediction in the back-end with MONOCULAR datasets, further details here)
  - in the front-end (to emulate an RGBD sensor) and a depth prediction/estimation gets available for each processed keyframe.

To obtain a mesh as output, set `kVolumetricIntegrationTsdfExtractMesh=True` in `config_parameters.py`.

Reload a saved sparse map and perform dense reconstruction

Use the script `main_map_dense_reconstruction.py` to reload a saved sparse map and perform dense reconstruction by using its posed keyframes as input. You can select your preferred dense reconstruction method directly in the script.

- To check what the volumetric integrator is doing, run in another shell `tail -f logs/volumetric_integrator.log` (from repository root folder).
- To save the obtained dense and sparse maps, press the `Save` button on the GUI.

Reload and check your dense reconstruction

You can check the output pointcloud/mesh by using CloudCompare.

In the case of a saved Gaussian splatting model, you can visualize it by:

1. Using the superslat editor (drag and drop the saved Gaussian splatting `.ply` pointcloud in the editor interface).
2. Getting into the folder `examples/gaussian_splatting` and running:

```
$ python example_gsm.py --load <gs_checkpoint_path>
```

The directory `<gs_checkpoint_path>` is expected to have the following structure:

```
+-- gs_checkpoint_path
|   +-- pointcloud          # folder containing different subfolders, each one with a saved .ply
|                           # encoding the Gaussian splatting model at a specific iteration/checkpoint
|   +-- last_camera.json
|   +-- config.yml
```

Controlling the spatial distribution of keyframe FOV centers

If you are targeting volumetric reconstruction while running SLAM, you can enable a **keyframe generation policy** designed to manage the spatial distribution of keyframe field-of-view (FOV) centers. The *FOV center of a camera* is defined as the backprojection of its image center, calculated using the median depth of the frame. With this policy, a new keyframe is generated only if its FOV center lies beyond a predefined distance from the nearest existing keyframe's FOV center. You can enable this policy by setting the following parameters in the yaml setting:

```
KeyFrame.useFovCentersBasedGeneration: 1  # compute 3D fov centers of camera frames by using median depth
                                          # use their distances to control keyframe generation
KeyFrame.maxFovCentersDistance: 0.2       # max distance between fov centers in order to generate a keyframe
```

or by setting the following parameters in `config_parameters.py`:

```
kUseFovCentersBasedKfGeneration = False  # Use FOV centers based keyframe generation;
                                         # not considered if KeyFrame.useFovCentersBasedGeneration is set in yaml
kMaxFovCentersDistanceForKfGeneration = 0.2  # [m] Maximum distance between FOV centers for keyframe generation;
                                         # not considered if not considered if KeyFrame.maxFovCentersDistance is set in yaml
```

Depth prediction

The available depth prediction models can be utilized both in the SLAM back-end and front-end.

- **Back-end**: Depth prediction can be enabled in the volumetric reconstruction pipeline by setting the parameter `kVolumetricIntegrationUseDepthEstimator=True` and selecting your preferred `kVolumetricIntegrationDepthEstimatorType` in `pyslam/config_parameters.py`.
- **Front-end**: Depth prediction can be enabled in the front-end by setting the parameter `kUseDepthEstimatorInFrontEnd` in `pyslam/config_parameters.py`. This feature estimates depth images from input color images to emulate a RGBD camera. Please, note this functionality is still *experimental* at present time.

Notes:

- In the case of a **monocular SLAM**, do NOT use depth prediction in the back-end volumetric integration: The SLAM (fake) scale will conflict with the absolute metric scale of depth predictions. With monocular datasets, you can enable depth prediction to run in the front-end (to emulate an RGBD sensor).
- Depth inference may be very slow (for instance, with DepthPro it takes ~1s per image on a typical machine). Therefore, the resulting volumetric reconstruction pipeline may be very slow.

Refer to the file `pyslam/depth_estimation/depth_estimator_factory.py` for further details. Both stereo and monocular prediction approaches are supported. You can test depth prediction/estimation by using the script `main_depth_prediction.py`.

Semantic mapping and Image Segmentation

The sparse semantic mapping pipeline can be enabled by setting the parameter `kDoSparseSemanticMappingAndSegmentation=True` in `pyslam/config_parameters.py`. You can configure the semantic mapping module by mofifying the default segmentation models assigned to each dataset in `pyslam/semantics/semantic_mapping_configs.py`. Otherwise, you can override the used segmentation model by setting `kSemanticSegmentationType` in `config_parameters.py`.

Sparse Semantic Mapping

Different semantic mapping methods are available (see here for further details). Currently, we support semantic mapping using **dense semantic segmentation**.

- `DEEPLABV3`: from `torchvision`, pre-trained on COCO/VOC.
- `SEGFORMER`: from `transformers`, pre-trained on Cityscapes or ADE20k.
- `CLIP`: from `f3rm` package for open-vocabulary support.
- `EOV_SEG`: open-vocabulary panoptic segmentation model.
- `DETIC`: open-vocabulary object detection and segmentation model.
- `ODISE`: open-vocabulary diffusion-based panoptic segmentation model.
- `RFDETR`: real-time transformer architecture for object detection and instance segmentation (built on a DINOv2 vision transformer backbone).
- `YOLO`: built on years of foundational research in computer vision and AI.

Panoptic/instance segmentation notes:

- `DETIC`: object detection-based. Each detection yields a distinct instance ID.
- `ODISE`: panoptic segmentation model. Instances are derived from panoptic segments via the `isthing` flag and can merge multiple objects of the same category.
- `EOV_SEG`: similar to `ODISE`, instances are derived from panoptic segments and may group nearby objects of the same category.

Semantic features are assigned to **keypoints** on the image and fused into map points. The semantic features can be:

- *Labels*: categorical labels as numbers.
- *Probability vectors*: probability vectors for each class.
- *Feature vectors*: feature vectors obtained from an encoder. This is generally used for open vocabulary mapping.

Volumetric Semantic mapping

Volumetric semantic mapping can be enabled by setting:

```
kDoSparseSemanticMappingAndSegmentation=True #  enable sparse mapping and segmentation
kDoVolumetricIntegration = True # enable volumetric integration
kVolumetricIntegrationType = "VOXEL_SEMANTIC_PROBABILISTIC_GRID" # use semantic volumetric models
                                                 # like VOXEL_SEMANTIC_PROBABILISTIC_GRID and VOXEL_SEMANTIC_GRID
```

Two semantic fusion backends are supported:

- `VOXEL_SEMANTIC_GRID`: confidence-counter fusion (majority voting).
- `VOXEL_SEMANTIC_PROBABILISTIC_GRID`: probabilistic fusion, with per-class distributions fused in log-space.

Configuration notes:

- If `kDoSparseSemanticMappingAndSegmentation=True` and `kVolumetricIntegrationType="VOXEL_GRID"`, the factory warns and automatically switches to `VOXEL_SEMANTIC_PROBABILISTIC_GRID` to enable semantic fusion.
- If `kDoSparseSemanticMappingAndSegmentation=True` with a non-semantic volumetric integrator (e.g., `TSDF`, `GAUSSIAN_SPLATTING`), a warning is emitted and semantic integration is skipped.

If `kVolumetricSemanticIntegrationUseInstanceIds=True`, the volumetric integrator uses 2D instance IDs (from panoptic/instance segmentation backends) to build **3D object segments**. Object IDs are assigned via voting across observations and filtered by `kVolumetricSemanticIntegrationMinVoteRatio` and `kVolumetricSemanticIntegrationMinVotes`. This enables:

- Consistent 3D object grouping across views.
- Per-object colorization of the volumetric map.
- Improved object identity stability over time.

When volumetric semantic mapping is enabled, use the following GUI buttons:

- `Color Semantics`: class/label color maps on both the 3D sparse map and the volumetric map.
- `Objects`: per-object color map on the volumetric map (requires instance IDs).
- `Draw Object BBs`: bounding boxes for detected 3D object segments.

Known limitations in volumetric semantic mapping:

- *Resolution limits*: a fixed voxel size may blur thin structures and small objects.
- *Pose drift sensitivity*: accumulated pose errors can smear semantic labels over time.
- *Instance ID dependence*: inconsistent or noisy 2D instance IDs may lead to fragmented or unstable 3D objects.
- *Class imbalance bias*: frequently observed classes may dominate voting/probabilistic fusion.
- *Temporal class inconsistency*: label flickering can fragment voxel labeling over time.
- *Occlusions and visibility gaps*: poorly observed regions may remain unlabeled or misclassified.

Further information about the **volumetric integration models and SW architecture** are available here.

A.7 Saving and reloading

Save a map

When you run the script `main_slam.py` (`main_map_dense_reconstruction.py`):

- You can save the current map state by pressing the button `Save` on the GUI. This saves the current map along with front-end, and backend configurations into the default folder `results/slam_state` (`results/slam_state_dense_reconstruction`).
- To change the default saving path, open `config.yaml` and update target `folder_path` in the section:

```
SYSTEM_STATE:
  folder_path: results/slam_state   # default folder path (relative to repository root) where the
                                    # system state is saved or reloaded
```

Reload a saved map and relocalize in it

- A saved map can be loaded and visualized in the GUI by running:

```
$ . pyenv-activate.sh   #  Activate pyslam python virtual environment. This is only needed once in a new terminal.
$ ./main_map_viewer.py  #  Use the --path options to change the input path
```

- To enable map reloading and relocalization when running `main_slam.py`, open `config.yaml` and set

```
SYSTEM_STATE:
  load_state: True                  # flag to enable SLAM state reloading (map state + loop closing state)
  folder_path: results/slam_state   # default folder path (relative to repository root) where the
                                    # system state is saved or reloaded
```

Note that pressing the `Save` button saves the current map, front-end, and backend configurations. Reloading a saved map replaces the current system configurations to ensure descriptor compatibility.

Trajectory saving

Estimated trajectories can be saved in three formats: *TUM* (The Open Mapping format), *KITTI* (KITTI Odometry format), and *EuRoC* (EuRoC MAV format). pySLAM saves two **types** of trajectory estimates:

- **Online**: In *online* trajectories, each pose estimate depends only on past poses. A pose estimate is saved at the end of each front-end iteration for the current frame.
- **Final**: In *final* trajectories, each pose estimate depends on both past and future poses. A pose estimate is refined multiple times by LBA windows that include it, as well as by PGO and GBA during loop closures.

To enable trajectory saving, open `config.yaml` and search for the `SAVE_TRAJECTORY`: set `save_trajectory: True`, select your `format_type` (`tum`, `kitti`, `euroc`), and the output filename. For instance for a `tum` format output:

```
SAVE_TRAJECTORY:
  save_trajectory: True
  format_type: kitti             # supported formats: `tum`, `kitti`, `euroc`
  output_folder: results/metrics # relative to pyslam root folder
  basename: trajectory           # basename of the trajectory saving output
```

Optimization engines

Currently, pySLAM supports both `g2o` and `gtsam` for graph optimization, with `g2o` set as the default engine. You can enable `gtsam` by setting to `True` the following parameters in `config_parameters.py`:

```
    # Optimization engine
    kOptimizationFrontEndUseGtsam = True
    kOptimizationBundleAdjustUseGtsam = True
    kOptimizationLoopClosingUseGtsam = True
```

Additionally, the `gtsam_factors` package provides custom Python bindings for features not available in the original gtsam framework. See here for further details.

A.8 SLAM GUI

Some quick information about the non-trivial GUI buttons of `main_slam.py`:

- `Step`: Enter the *Step by step mode*. Press the button `Step` a first time to pause. Then, press it again to make the pipeline process a single new frame.

- `Save`: Save the map into the file `map.json`. You can visualize it back by using the script `main_map_viewer.py` (as explained above).
- `Reset`: Reset SLAM system.
- `Draw Ground Truth`: If a ground truth dataset (e.g., KITTI, TUM, EUROC, or REPLICA) is loaded, you can visualize it by pressing this button. The ground truth trajectory will be displayed in 3D and will be progressively aligned with the estimated trajectory, updating approximately every 10-30 frames. As more frames are processed, the alignment between the ground truth and estimated trajectory becomes more accurate. After about 20 frames, if the button is pressed, a window will appear showing the Cartesian alignment errors along the main axes (i.e., e_x, e_y, e_z and the history of the total $RMSE$ between the ground truth and the aligned estimated trajectories.

A.9 Monitor the logs for tracking, local mapping, and loop closing simultaneously

The logs generated by the modules `local_mapping.py`, `loop_closing.py`, `loop_detecting_process.py`, `global_bundle_adjustments.py`, and `volumetric integrator_<X>.py` are collected in the files `local_mapping.log`, `loop_closing.log`, `loop_detecting.log`, `gba.log`, and `volumetric_integrator.log`, which are all stored in the folder `logs`.

For debugging, you can monitor one of the them in parallel by running the following command in a separate shell:

```
$ tail -f logs/<log file name>
```

Otherwise, to check all parallel logs with tmux, run:

```
$ ./scripts/launch_tmux_logs.sh
```

To launch slam and check all logs in a single tmux, run:

```
$ ./scripts/launch_tmux_slam.sh
```

Press `CTRL+A` followed by `CTRL+Q` to exit from `tmux` environment.

A.10 Scene from Views

The *Scene from Views* module provides a unified interface for feed-forward 3D scene reconstruction from multiple views, following a modular factory pattern architecture. The pipeline provides a consistent API across different reconstruction models while maintaining model-specific optimizations. See the main script `main_scene_from_views.py`.

- Feed-forward multi-view reconstruction (poses + fused 3D geometry directly from images): `SceneFromViewsDust3r`, `SceneFromViewsMast3r`, `SceneFromViewsMvdust3r`, `SceneFromViewsVggt`, `SceneFromViewsVggtRobust`, `SceneFromViewsFast3r`.
- Single-view depth-first pipeline with optional poses/intrinsics: `SceneFromViewsDepthAnythingV3`
- Global alignment optimization stage for merging views: `SceneFromViewsDust3r` (dense alignment) and `SceneFromViewsMast3r` (sparse alignment variant)
- Robust view filtering / outlier rejection: `SceneFromViewsVggtRobust` (anchor-based attention + cosine scoring that discards low-confidence views before reconstruction)

Note that `DUSt3R` and `MASt3R` are pairwise models: they take two images at a time. They are feed-forward per pair; multi-view reconstruction is then obtained by global alignment / optimization over all pairwise pointmaps. `MV-DUSt3R`, `VGGT`, and `Fast3R` are multi-view models that process all images simultaneously in a single forward pass.

All models return standardized `SceneFromViewsResult` output containing point clouds, meshes, camera poses, depth maps, and intrinsics in a consistent format. The factory pattern allows easy switching between models while maintaining the same interface.

Further details here.

B Configuration

B.1 Main configuration file

Refer to this section for how to update the main configuration file config.yaml and affect the configuration parameters in config_parameters.py.

B.2 Datasets

The following datasets are supported:

Dataset	type in `config.yaml`
KITTI odometry data set [35]	`type: KITTI_DATASET`
TUM dataset [95]	`type: TUM_DATASET`
TUM-VI dataset [90]	`type: TUM_VI_DATASET`
ICL-NUIM dataset [39]	`type: ICL_NUIM_DATASET`
EUROC dataset [19]	`type: EUROC_DATASET`
REPLICA dataset [94]	`type: REPLICA_DATASET`
TARTANAIR dataset [113]	`type: TARTANAIR_DATASET`
Seven Scenes dataset [37]	`type: SEVEN_SCENES_DATASET`
Neural RGBD dataset [7]	`type: NEURAL_RGBD_DATASET`
Rover dataset [89]	`type: ROVER_DATASET`
ScanNet dataset [25]	`type: SCANNET_DATASET`
CLIO dataset [47]	`type: CLIO_DATASET`
ROS1 bags	`type: ROS1BAG_DATASET`
ROS2 bags	`type: ROS2BAG_DATASET`
MCAP file	`type: MCAP_DATASET`
Video file	`type: VIDEO_DATASET`
Folder of images	`type: FOLDER_DATASET`

Use the download scripts available in the folder `scripts` to download some of the following datasets.

File-based datasets prefetch decoded images in a background thread, using the optional C++ `data_thread_reader` module, so that image decoding overlaps with tracking. Prefetching can be disabled with the environment variable `PYSLAM_DISABLE_IMAGE_PREFETCH=1`. Supported readers, build instructions, and memory behavior are described here.

KITTI Datasets

pySLAM code expects the following structure in the specified KITTI path folder (specified in the section `KITTI_DATASET` of the file `config.yaml`). :

```
+-- sequences
|   +-- 00
|   ...
|   +-- 21
+-- poses
    +-- 00.txt
    ...
    +-- 10.txt
```

1. Download the dataset (grayscale images) from http://www.cvlibs.net/datasets/kitti/eval_odometry.php and prepare the KITTI folder as specified above
2. Select the corresponding calibration settings file (section `KITTI_DATASET: settings:` in the file `config.yaml`)

TUM Datasets

pySLAM code expects a file `associations.txt` in each TUM dataset folder (specified in the section `TUM_DATASET:` of the file `config.yaml`).

1. Download a sequence from vision.in.tum.de/data/datasets/rgbd-dataset/download and uncompress it.
2. Associate RGB images and depth images using the python script associate.py. You can generate your `associations.txt` file by executing:
   ```
   $ python associate.py PATH_TO_SEQUENCE/rgb.txt PATH_TO_SEQUENCE/depth.txt > associations.txt
   # pay attention to the order!
   ```
3. Select the corresponding calibration settings file (section `TUM_DATASET: settings:` in the file `config.yaml`).

TUM-VI Datasets

pySLAM reads the 512×512 image sequences of TUM-VI [90]; IMU measurements are not used. Each sequence folder (the `name` field of the section `TUM_VI_DATASET` in `config.yaml`) is expected to contain `mav0/cam0/data.csv`, `mav0/cam0/data/<timestamp>.png`, the same two entries for `cam1`, and `mav0/mocap0/data.csv`.

1. Download a 512×512 sequence from cvg.cit.tum.de/data/datasets/visual-inertial-dataset and uncompress it under `TUM_VI_DATASET: base_path`.
2. Set `DATASET.type: TUM_VI_DATASET`. Set `sensor_type` to `stereo` or `mono`, and `stereo_mode` to `native` (original Kannala–Brandt images) or `rectified` (fisheye images remapped to a pinhole stereo pair). For monocular fisheye tracking use `sensor_type: mono` with `stereo_mode: native`. See Sect. 4.4 for the differences between the two modes.
3. Select `settings/TUM_VI.yaml`. With `groundtruth_file: auto`, the motion-capture poses are converted to the tracking camera frame. Calibration, rectification, and ground-truth conversion are detailed here.

ICL-NUIM Datasets

Follow the same instructions provided for the TUM datasets.

EuRoC Datasets

1. Download a sequence (ASL format) from http://projects.asl.ethz.ch/datasets/doku.php?id=kmavvisualinertialdatasets (check this direct link)
2. Use the script `io/generate_euroc_groundtruths_as_tum.sh` to generate the TUM-like groundtruth files `path + '/' + name + '/mav0/state_groundtruth_estimate0/data.tum'` that are required by the `EurocGroundTruth` class.
3. Select the corresponding calibration settings file (section `EUROC_DATASET: settings:` in the file `config.yaml`).

Replica Datasets

1. You can download the zip file containing all the sequences by running:
   `$ wget https://cvg-data.inf.ethz.ch/nice-slam/data/Replica.zip`

2. Then, uncompress it and deploy the files as you wish.
3. Select the corresponding calibration settings file (section `REPLICA_DATASET: settings:` in the file `config.yaml`).

Tartanair Datasets

1. You can download the datasets from https://theairlab.org/tartanair-dataset
2. Then, uncompress them and deploy the files as you wish.
3. Select the corresponding calibration settings file (section `TARTANAIR_DATASET: settings:` in the file `config.yaml`).

7 Scenes dataset

To dowload the dataset follow the instructions at https://www.microsoft.com/en-us/research/project/rgb-d-dataset-7-scenes/. Select the section `SEVEN_SCENES_DATASET: settings:` in the file `config.yaml`.A calibration file `settings/SEVEN_SCENES.yaml` is already available.

Neural RGBD dataset

To dowload the dataset follow the instructions at https://github.com/dazinovic/neural-rgbd-surface-reconstruction. Select the section `NEURAL_RGBD_DATASET: settings:` in the file `config.yaml`. A calibration file `settings/NEURAL_RGBD.yaml` is already available.

Rover dataset

To dowload the dataset follow the instructions at https://iis-esslingen.github.io/rover/. Select the section `ROVER_DATASET: settings:` in the file `config.yaml`. A calibration file for the `d435i` camera is already available: `settings/ROVER_d435i.yaml`. Other camera models will be supported in the future.

ScanNet Datasets

1. You can download the datasets following instructions in http://www.scan-net.org. You will need to request the dataset from the authors.
2. There are two versions you can download:
   - A subset of pre-processed data termed as `tasks/scannet_frames_2k`: this version is smaller, and more generally available for training neural networks. However, it only includes one frame out of each 100, which makes it unusable for SLAM. The labels are processed by mapping them from the original Scannet label annotations to NYU40.
   - The raw data: this version is the one used for SLAM. You can download the whole dataset (TBs of data) or specific scenes. A common approach for evaluation of semantic mapping is to use the `scannetv2_val.txt` scenes. For downloading and processing the data, you can use the following repository as the original Scannet repository is tested under Python 2.7 and does't support batch downloading of scenes.
3. Once you have the `color`, `depth`, `pose`, and (optional for semantic mapping) `label` folders, you should place them following `{path_to_scannet}/scans/{scene_name}/[color, depth, pose, label]`. Then, configure the `base_path` and `name` in the file `config.yaml`.
4. Select the corresponding calibration settings file (section `SCANNET_DATASET: settings:` in the file `config.yaml`). NOTE: the RGB images are rescaled to match the depth image. The current intrinsic parametes in the existing calibration file reflect that.

CLIO Datasets

pySLAM supports the RGB-D sequences released with Clio [47] (Hydra/Clio scene-graph mapping). Each scene is expected to follow this layout under `{base_path}/{name}/`:

```
images/rgb_{id}.jpg
depth/depth_{id}.png
sparse/0/images.bin          # COLMAP sparse model (optional reference poses)
*.bag                        # ROS1 recording (fps auto-detection, metric odometry)
```

1. Download or prepare a CLIO scene and set `base_path` and `name` in the `CLIO_DATASET` section of `config.yaml`.
2. Select the calibration settings file (`CLIO_DATASET: settings: settings/CLIO.yaml`). Depth maps are stored as `uint16` in **millimeters** (`DepthMapFactor: 1000.0`).
3. Set `sensor_type: rgbd` (or `mono` if depth is unavailable).
4. Set `groundtruth_file: auto` to load a **reference trajectory** for evaluation and visualization (not independent motion-capture GT). By default, pySLAM prefers metric poses from the scene ROS1 bag topic `/dominic/forward/colmap_odom`; if the bag is missing, it falls back to COLMAP `sparse/0` poses (typically up-to-scale). Use `clio_reference_poses: auto | bag | sparse` to override this choice.
5. Playback rate is auto-detected from the bag when present (∼7.5 Hz on the office scene); you can override it with `fps` in `config.yaml` or `Camera.fps` in the settings file.

When the reference trajectory is non-metric (COLMAP sparse), pySLAM automatically uses Sim(3) alignment for the ground-truth overlay and ATE metrics.

ROS1 bags

1. Source the main ROS1 `setup.bash` after you have sourced the `pyslam` python environment.
2. Set the paths and `ROS1BAG_DATASET: ros_parameters` in the file `config.yaml`.
3. Select/prepare the correspoding calibration settings file (section `ROS1BAG_DATASET: settings:` in the file `config.yaml`). See the available yaml files in the folder `Settings` as an example.

ROS2 bags

1. Source the main ROS2 `setup.bash` after you have sourced the `pyslam` python environment.
2. Set the paths and `ROS2BAG_DATASET: ros_parameters` in the file `config.yaml`.
3. Select/prepare the correspoding calibration settings file (section `ROS2BAG_DATASET: settings:` in the file `config.yaml`). See the available yaml files in the folder `Settings` as an example.

MCAP file

A concise introduction of the MCAP format is available at https://foxglove.dev/blog/introducing-the-mcap-file-format. These files can be also recorded and played back by using ROS2 tools.

Video and Folder Datasets

You can use the `VIDEO_DATASET` and `FOLDER_DATASET` types to read generic video files and image folders (specifying a glob pattern), respectively. A companion ground truth file can be set in the simple format type: Refer to the

class `SimpleGroundTruth` in `io/ground_truth.py` and check the script `io/convert_groundtruth_to_simple.py`.

B.3 Camera Settings

The folder `settings` contains the camera settings files which can be used for testing the code. These are the same used in the framework ORB-SLAM2 [72]. You can easily modify one of those files for creating your own new calibration file (for your new datasets).

A settings file that does not set `Camera.type` uses the linear pinhole model: `Camera.k1`, . . . , `Camera.k3` are used only to undistort keypoints. Setting `Camera.type` to `kannala_brandt4`, `pinhole_radtan8`, or `double_sphere` selects the corresponding model of Sect. 4.4. Note that `Camera.k3` and `Camera.k4` are the higher-order polynomial coefficients in a Kannala–Brandt file, not the tangential coefficients. A non-pinhole stereo rig requires an explicit right camera (`Stereo.right`) and the relative pose `Stereo.T_right_left`. Parameters and pixel conventions of each model are detailed here.

In order to calibrate your camera, you can use the scripts in the folder `calibration`. In particular: 1. Use the script `grab_chessboard_images.py` to collect a sequence of images where the chessboard can be detected (set the chessboard size therein, you can use the calibration pattern `calib_pattern.pdf` in the same folder) 2. Use the script `calibrate.py` to process the collected images and compute the calibration parameters (set the chessboard size therein)

For more information on the calibration process, see this tutorial [64] or this other link [78].

If you want to **use your camera**, you have to:

- Calibrate it and configure settings/WEBCAM.yaml accordingly.
- Record a video (for instance, by using `save_video.py` in the folder `calibration`).
- Configure the `VIDEO_DATASET` section of `config.yaml` in order to point to your recorded video.